GANs – Part 1: Generation

Author: Sebastian Bujwid bujwid@kth.se

Refer to ./README.md for general comments.

Scope

- Implementing a GAN model
- Training a GAN on two toy 2-D datasets:
  - Guassian blobs
  - Swiss roll

Objectives

The point of this part of the practical is to get yourself familiar with a GAN model. You are expected to learn technical implementation details of a GAN but also get a better feeling of the challenges of GAN training. As this practical uses simple 2-D data, it will be easy to understand what the model does and what it has learned.

GAN

A GAN model introduced by Goodfellow *et al.* (2014) consists of two networks trained in adversary, a generator and a discriminator. The generator's goal is to produce realistic samples while the discriminator tries to tell apart the real data samples from the fake samples produced by the generator. The better the generator gets at generating samples, the more difficult discriminator's job is.

Note that as GAN contain two separate networks, each trained to be better than the other, the training dynamics can be quite different from standard supervised training. It's not uncommon to observe the losses go up and down during training, as one of the networks get better than the other.

Technical details

This Notebook has some `assert`s or checks which you can use to test your implementation of some of the functions. However, keep in mind that passing those checks does not necessarily mean that everything is correct!

Some of the `assert`s verify whether shapes of tensors are as expected. Those shapes, however, do not necessarily have to be as expected to have a correct implementation. You are, of course, free to do things differently but make sure you know what you're doing!

```
import numpy as np

import sklearn.datasets
import matplotlib.pyplot as plt
import seaborn as sns
import itertools
```

```
seed = 42
```

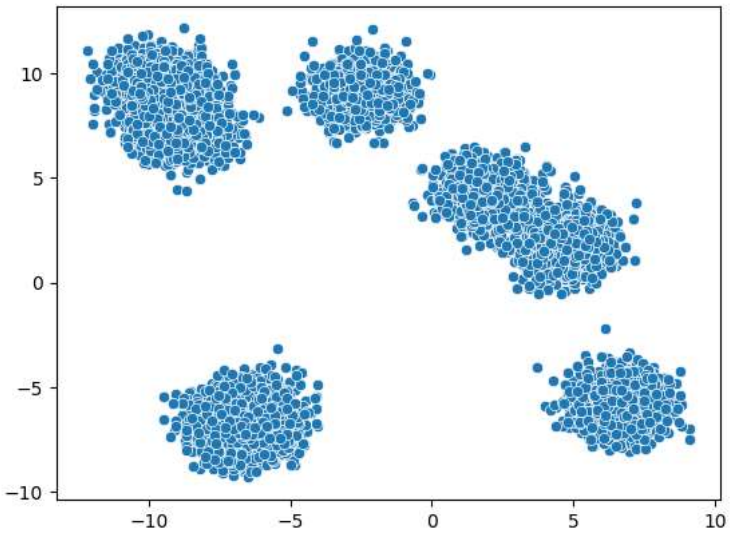
Toy datasets

```
def dataset_swiss_roll(n_samples, seed):
    x, _ = sklearn.datasets.make_swiss_roll(n_samples, noise=0.5, random_state=seed)
    x = x[:, [0, 2]]
    return x / np.max(np.abs(x)) + np.array([0., 5.])


def dataset_s_curve(n_samples, seed):
    x, _ = sklearn.datasets.make_s_curve(n_samples, noise=0.15, random_state=seed)
    x = x[:, [0, 2]]
    return x


def dataset_blobs(n_samples, seed):
    x, _ = sklearn.datasets.make_blobs(
        n_samples=n_samples, n_features=2,
        centers=8, cluster_std=0.8,
        random_state=seed)
    return x


x = dataset_blobs(n_samples=10000, seed=seed)
sns.scatterplot(x=x[:, 0], y=x[:, 1])
plt.show()
```



Loss functions

```
import jax
import jax.numpy as jnp
from jax.example_libraries import optimizers, stax
from jax import jit
```

If you have a GPU, make sure you're using it!

```
jax.devices()
```

```
[gpu(id=0)]
```

(Task) Implement `sigmoid_cross_entropy` function

First, we'll implement a `sigmoid_cross_entropy` function that transforms the input logits with the sigmoid function and computes the cross-entropy loss.

The function should compute the following:

$$loss(t, l) = -\left[t \cdot \log S(l) + (1 - t) \cdot \log(1 - S(l))\right]$$

where `t` represents the input target value; `l` the logit; and `S(x)` the sigmoid function:

$$S(x) = \frac{1}{1 + e^{-x}}$$

Note, that the sigmoid & cross-entropy can be thought of as separate function but here we expect it to be a single function.

Question:

- Is there any (objective) potential benefit of implementing the combination of sigmoid & cross-entropy as a single function?

```
def sigmoid_cross_entropy(*, targets, logits):
    assert targets.shape == logits.shape


    sigmoid_logits = jax.nn.sigmoid(logits)
    loss = - (targets * jnp.log(sigmoid_logits) + (1 - targets) * jnp.log(1 - sigmoid_logits))


    #raise NotImplementedError('Task: implement!')
    #loss = NotImplemented

    assert loss.shape == logits.shape, \
        'cross-entropy loss is not expected here to be averaged over samples'
    return loss


sigmoid_cross_entropy(targets=jnp.array(1.), logits=jnp.array(0.))
```

```
Array(0.6931472, dtype=float32, weak_type=True)
```

```
sigmoid_cross_entropy(targets=jnp.array([[1.], [1.]]), logits=jnp.array([[0.], [0.]]))
```

```
Array([[0.6931472],
       [0.6931472]], dtype=float32)
```

(Task) Test `sigmoid_cross_entropy`

```
targets = np.array([0, 1, 0, 1, 1])
logits = np.array([0.55, -0.22, 0.8, 0.1, -0.42])
returned = sigmoid_cross_entropy(targets=targets, logits=logits)
expected_mean_ce_loss = np.array(0.9110423)

#np.testing.assert_allclose(expected_mean_ce_loss, jnp.mean(returned))
```

(Optional task) Numerically stable `sigmoid_cross_entropy` loss

Feel free to skip this task (or maybe try it at the end after completing everything else?). You should (hopefully) be able to complete the rest of the practical even if your `sigmoid_cross_entropy` function is not able to handle very large or small values, producing NaNs. Note that you typically do not want your model to internally (hidden features, logits, etc.) operate on very large numbers. Such a behavior might indicate some "issues with training". However, it does not have to imply any major issues and sometimes might be even expected.

Numerical stability of implementation can be important and it is good to be at least familiar with what are the main roots of numerical stability issues and how to address them, what are the common "tricks".

(Optional) task:

- Can your function compute the value for $t = 0$ and $l = 10000.0$ or do you get NaNs?
- Do you understand what is the source of the numerical instability (NaNs)?
  - Hint: Think of computing $\log(e^x)$ for $x = 10^9$, for example. The value of the expressions is impossible to calculate by `np.log(np.exp(x))` (returns inf/NaN) but we know that $\log(e^x) = x$.
- Can you compute by hand the (almost?) exact value of the function for the inputs as above? **Try to transform the formula manually and see if you can implement a more numerically stable `sigmoid_cross_entropy` function that works with very large or small numbers.**
- Note that expressions in math are usefually written in the form that is the most readable or intuitive. However, that form might not necessarily be the best to be implemented. Not just because of numerical stability but also in terms of speed or memory usage.
- Hopefully, in the future you'd be able to foresee numerical stability issues and address them! The operations like `log` and `exp` you have to be especially careful about!

```
sigmoid_cross_entropy(targets=jnp.array(0.),
                      logits=jnp.array(10000.))
```

```
Array(inf, dtype=float32, weak_type=True)
```

GAN: objectives

In this practical, we're going to use the GAN training objectives as introduced by Goodfellow *et al.* (2014) (note that $D(x) \in (0, 1)$):

(discriminator)
$$-\mathbb{E}_{x\sim p_{\text{data}}(x)}\left[\log D(x)\right] - \mathbb{E}_{z\sim p_z(z)}\left[\log(1-D(G(z)))\right]$$

(generator: non-saturating objective)
$$-\mathbb{E}_{z\sim p_z(z)}\left[\log D(G(z))\right].$$

Note we use the **non-saturating** loss for the generator. The alternative is, of course, to have the generator optimize just the reverse of the discriminator's objective. In the non-saturating loss, instead of training the generator to minimize the probability of the discriminator being correct, we maximize the probability of the discriminator incorrectly classifying generated samples as true. Therefore, thanks to the formulation of non-saturating loss, the generator can learn (as it gets non-zero gradients) when the discriminator is very strong and correctly classifies generated samples as fake (that is when $log(1 - D(G(z))$ would be close to 0).

Alternatively, there is a number of modified objectives for training GANs, such as for example:

- WGAN
- WGAN-GP
- LSGAN

(Task) Implement discriminator's loss

Hint: use `sigmoid_cross_entropy` function

```
def discriminator_loss(*, real_logits, fake_logits):
    # NOTE: the inputs are expected to be logits, not probabilities!



    real_targets = jnp.ones_like(real_logits)  # Real targets are all 1
    fake_targets = jnp.zeros_like(fake_logits)  # Fake targets are all 0

    real_loss = jnp.mean(sigmoid_cross_entropy(targets=real_targets, logits=real_logits))
    fake_loss = jnp.mean(sigmoid_cross_entropy(targets=fake_targets, logits=fake_logits))

    # Compute the average loss
    loss = real_loss + fake_loss



    #raise NotImplementedError('Task: implement!')
    #loss = NotImplemented

    assert loss.shape == (), \
        'discriminator loss is expected here to be averaged over samples'
    return loss
```

What result would you expect if both real & fake input logits were equal 0.0?

Try out a couple of values! Does the function return what you would expect it to return? What if you set the real logits high and fake logits low? What if it's vice-versa?

```
discriminator_loss(real_logits=jnp.array([[0.], [0.]]), fake_logits=jnp.array([[0.], [0.]]))
```

```
Array(1.3862944, dtype=float32)
```

```
discriminator_loss(real_logits=jnp.array([0., 0.]), fake_logits=jnp.array([0., 0.]))
```

```
Array(1.3862944, dtype=float32)
```

(Task) Test the discriminator loss

```
returned = discriminator_loss(
    real_logits=jnp.array([0.13, -0.11, -1.12, -0.02]),
    fake_logits=jnp.array([-0.88, 0.42, 0.42, -0.81])
)
expected = 1.5126383
np.testing.assert_almost_equal(returned, expected)
print (returned)
print (expected)
```

```
1.5126383
1.5126383
```

(Task) Implement generator loss

Hint: Use `sigmoid_cross_entropy` function

```
def generator_loss(discriminator_fake_logits):
    # NOTE: the inputs are expected to be logits, not probabilities!

    #raise NotImplementedError('Task: implement!')
    #loss = NotImplemented
    loss = jnp.mean(sigmoid_cross_entropy(targets=jnp.ones_like(discriminator_fake_logits), logits=discriminator_fake_logits))


    assert loss.shape == (), \
        'discriminator loss is expected here to be averaged over samples'
    return loss
```

(Task) Testing the generator loss

What result would you expect for the output of the generator $G(z)$ (which is the input to $D(.)$) equal 0.0? Does the result match your expectation?

```
generator_loss(discriminator_fake_logits=jnp.array([[0.], [0.,]]))
```

```
Array(0.6931472, dtype=float32)
```

```
generator_loss(discriminator_fake_logits=jnp.array([0., 0.]))
```

```
Array(0.6931472, dtype=float32)
```

```
returned = generator_loss(discriminator_fake_logits=jnp.array([1.12, -0.12, 0., 0.42, -0.82,]))
expected = 0.68409014
np.testing.assert_allclose(expected, returned)
```

Training

```
#from utils import plot_losses
```

```
import matplotlib.pyplot as plt
```

```
def plot_losses(losses_dict):
    for key, vals in losses_dict.items():
        plt.plot(vals, label=key)
    plt.legend()
    plt.show()
```

```
def iterable_dataset(key, data, batch_size):
    n_batches = len(data) // batch_size
    x_shuffled = jax.random.permutation(key, data)
    dataset = [x_shuffled[i * batch_size: (i + 1) * batch_size] for i in range(n_batches)]
    return dataset
```

```
def plot_losses(losses_dict):
    for key, vals in losses_dict.items():
        plt.plot(vals, label=key)
    plt.legend()
    plt.show()
```

```
def generate_and_plot_samples(key, real_data, generator_apply, gen_params, n_samples, input_noise_dim):
    noise = jax.random.normal(key, (n_samples, input_noise_dim))
    samples = generator_apply(gen_params, noise)
    sns.scatterplot(x=real_data[:, 0], y=real_data[:, 1], label='real data')
    sns.scatterplot(x=samples[:, 0], y=samples[:, 1], label='generated samples')
    plt.legend()
    plt.show()
```

```
pip install omegaconf
```

```
Collecting omegaconf
  Downloading omegaconf-2.3.0-py3-none-any.whl (79 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 79.5/79.5 kB 2.2 MB/s eta 0:00:00
Collecting antlr4-python3-runtime==4.9.* (from omegaconf)
  Downloading antlr4-python3-runtime-4.9.3.tar.gz (117 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 117.0/117.0 kB 7.4 MB/s eta 0:00:00
  Preparing metadata (setup.py) ... done
Requirement already satisfied: PyYAML>=5.1.0 in /usr/local/lib/python3.10/dist-packages (from omegaconf) (6.0.1)
Building wheels for collected packages: antlr4-python3-runtime
  Building wheel for antlr4-python3-runtime (setup.py) ... done
  Created wheel for antlr4-python3-runtime: filename=antlr4_python3_runtime-4.9.3-py3-none-any.whl size=144554 sha256=d5f3f6962c6c88aca77728b7e0d9109beea9198132521306a960e9c20b5f7083
  Stored in directory: /root/.cache/pip/wheels/12/93/dd/1f6a127edc45659556564c5730f6d4e300888f4bca2d4c5a88
Successfully built antlr4-python3-runtime
Installing collected packages: antlr4-python3-runtime, omegaconf
```

```
from omegaconf import OmegaConf
```

```
[pydevd plugins]
```

```
def create_mlp(n_hid_layers, hid_dim, output_dim):
    layers = list(itertools.chain.from_iterable(
        [(stax.Dense(hid_dim), stax.Relu) for _ in range(n_hid_layers)]
    )) + [stax.Dense(output_dim)]
    mpl_init, mpl_apply = stax.serial(*layers)
    return mpl_init, mpl_apply
```

(Task) Implement the `train_step` function

Note that the generator and discriminator's parameters should be updated individually. Make sure that the generator's gradients and discriminator's gradients depend on the right "things".

```
def create_and_train_gan(hparams, data, seed=seed):

    data_dim = data[0].shape[-1]

    # Create models

    if hparams.model_type == 'mlp':
        generator_init, generator_apply = create_mlp(
            n_hid_layers=hparams.generator.n_layers,
            hid_dim=hparams.generator.hidden_dim,
            output_dim=data_dim,
        )
        discriminator_init, discriminator_apply = create_mlp(
            n_hid_layers=hparams.discriminator.n_layers,
            hid_dim=hparams.discriminator.hidden_dim,
            output_dim=1,
        )
    else:
        raise NotImplementedError(f'model_type: {hparams.model_type}')




    # Initialize
```

```
key = jax.random.PRNGKey(seed)
key, key_gen, key_dis = jax.random.split(key, 3)

input_noise_shape = (-1, hparams.input_noise_dim)
data_shape = (-1, data_dim)
_, gen_params = generator_init(key_gen, input_noise_shape)
_, dis_params = discriminator_init(key_dis, data_shape)

## Initialize: optimizers
gen_opt_init, gen_opt_update, gen_get_params = optimizers.adam(
    step_size=hparams.gen_lr, b1=hparams.beta1)
dis_opt_init, dis_opt_update, dis_get_params = optimizers.adam(
    step_size=hparams.dis_lr, b1=hparams.beta1)
gen_opt_state = gen_opt_init(gen_params)
dis_opt_state = dis_opt_init(dis_params)


# NOTE: can comment out @jax.jit for debugging but
#       make sure to use it training as it will be much faster
@jax.jit
def train_step(step, gen_opt_state, dis_opt_state, noise, real_samples):

    gen_params = gen_get_params(gen_opt_state)
    dis_params = dis_get_params(dis_opt_state)

    # Generator loss and gradients
    def gen_loss_fn(params, noise):
        fake_samples = generator_apply(params, noise)
        fake_logits = discriminator_apply(dis_params, fake_samples)
        return generator_loss(fake_logits)

    gen_grads = jax.grad(gen_loss_fn)(gen_params, noise)
    gen_opt_state = gen_opt_update(step, gen_grads, gen_opt_state)


    def dis_loss_fn(params, real_samples, noise):
        fake_samples = generator_apply(gen_params, noise)
        real_logits = discriminator_apply(params, real_samples)
        fake_logits = discriminator_apply(params, fake_samples)
        return discriminator_loss(real_logits=real_logits,fake_logits=fake_logits)

    dis_grads = jax.grad(dis_loss_fn)(dis_params, real_samples, noise)
    dis_opt_state = dis_opt_update(step, dis_grads, dis_opt_state)


    #raise NotImplementedError('Task: implement!')
    #gen_grads = NotImplemented
    #dis_grads = NotImplemented
    #gen_loss = NotImplemented
    #dis_loss = NotImplemented

    #gen_opt_state = gen_opt_update(step, gen_grads, gen_opt_state)



    gen_loss = gen_loss_fn(gen_params, noise)
    dis_loss = dis_loss_fn(dis_params, real_samples, noise)
```

```
        dis_loss = dis_loss_fn(dis_params, real_samples, noise)

        return (gen_loss, dis_loss), (gen_opt_state, dis_opt_state)



    # Training
    losses = {'gen': [], 'dis': []}

    total_step = 0
    for epoch in range(hparams.epochs):
        key, key_data = jax.random.split(key)

        for batch_x in iterable_dataset(key=key_data, data=data, batch_size=hparams.batch_size):

            key, subkey = jax.random.split(key)
            noise = jax.random.normal(subkey, (hparams.batch_size, hparams.input_noise_dim))

            (gen_loss, dis_loss), (gen_opt_state, dis_opt_state) = train_step(
                step=total_step,
                gen_opt_state=gen_opt_state,
                dis_opt_state=dis_opt_state,
                noise=noise,
                real_samples=batch_x,
            )
            losses['gen'].append(gen_loss)
            losses['dis'].append(dis_loss)

            total_step += 1

        if epoch == 0 or (epoch < 100 and epoch % 10 == 9) or epoch % 100 == 99:
            print('-' * 30, 'epoch', epoch, '-' * 30)
            plot_losses(losses)
            key, subkey = jax.random.split(key)
            generate_and_plot_samples(
                subkey, data,
                generator_apply, gen_get_params(gen_opt_state),
                n_samples=256, input_noise_dim=hparams.input_noise_dim)
```

(Task) Train the GAN model: simple blobs data

Below you can the output of my training. Can you get similar results? Note that the output you get might not be exactly the same as the one below but given that the code skeleton already covers most of the details that could matter, it should be fairly similar.

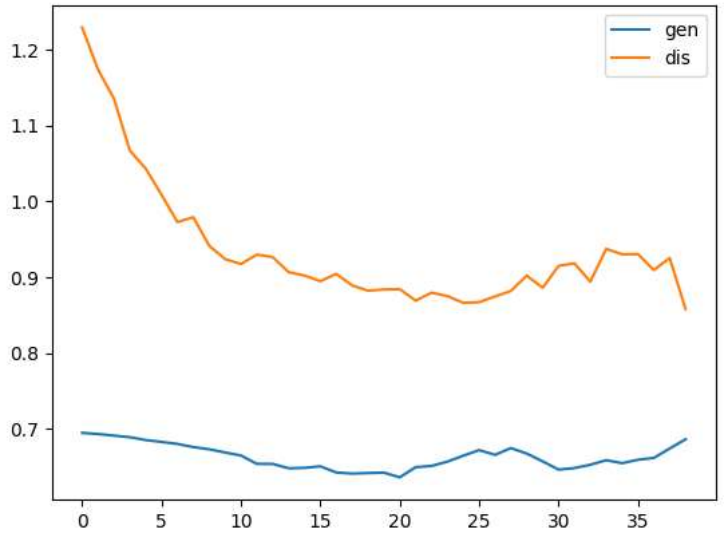
Questions:

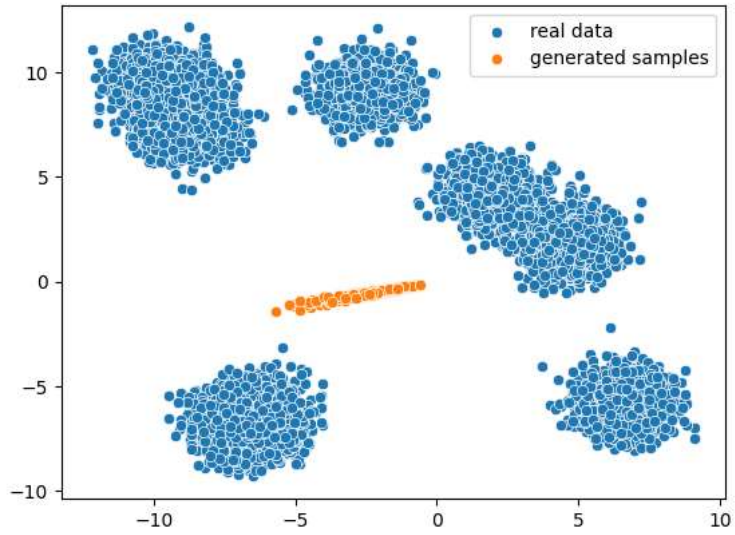
- Is the generator able to cover all of the blobs?
- Are the individual blobs well covered by the generated samples?
- Can you observe mode collapse?

```
hparams = OmegaConf.create({
    'epochs': 1000,
    'batch_size': 256,
```

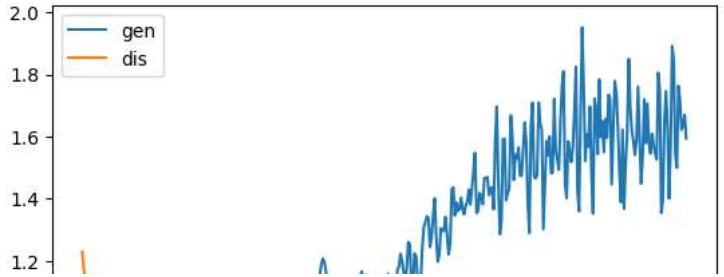
```
        'gen_lr': 0.0001,
        'dis_lr': 0.0001,
        'beta1': 0.5,  # NOTE: important!
        'model_type': 'mlp',
        'generator': {
            'n_layers': 3,
            'hidden_dim': 512,
        },
        'discriminator': {
            'n_layers': 2,
            'hidden_dim': 512,
        },
        'input_noise_dim': 2,
    })
    create_and_train_gan(hparams, dataset_blobs(n_samples=10000, seed=seed), seed=seed)
```

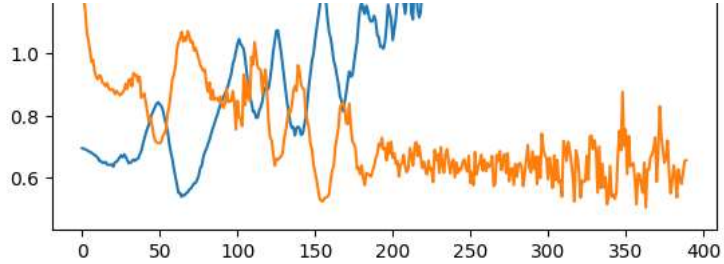
```
------------------------------ epoch 0 ------------------------------
```

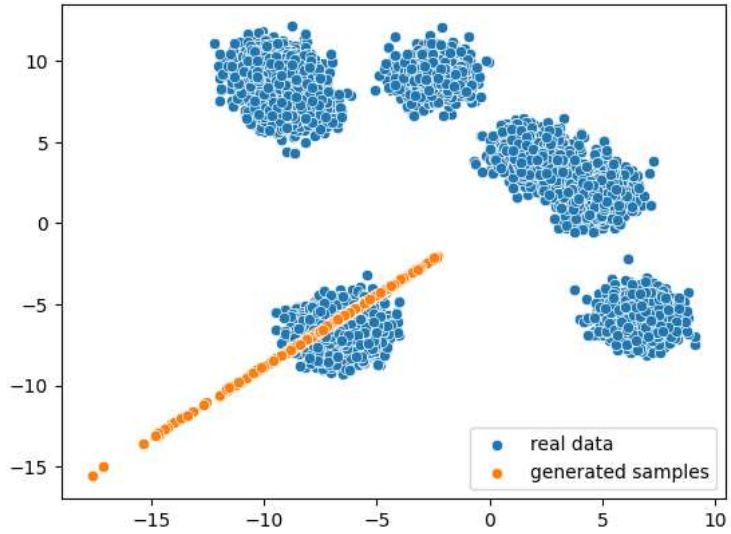




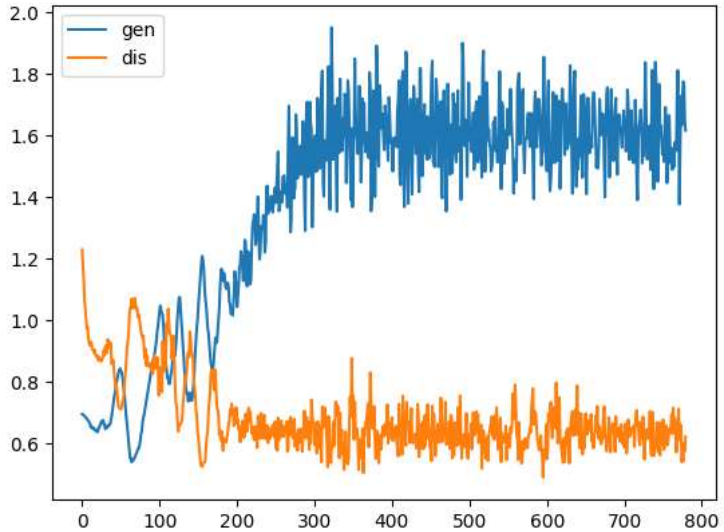

```
------------------------------ epoch 9 ------------------------------
```



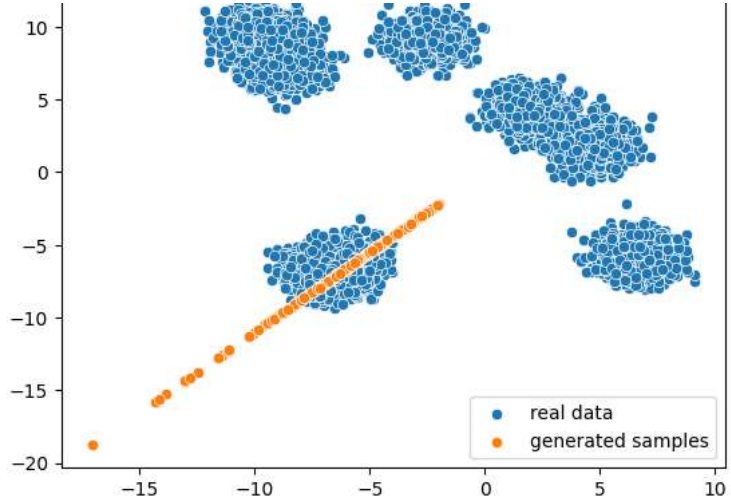

1.0
0.8
0.6
0
50
100
150
200
250
300
350
400


10
5
0
−5
−10
−15
−15
−10
−5
0
5
10
real data
generated samples

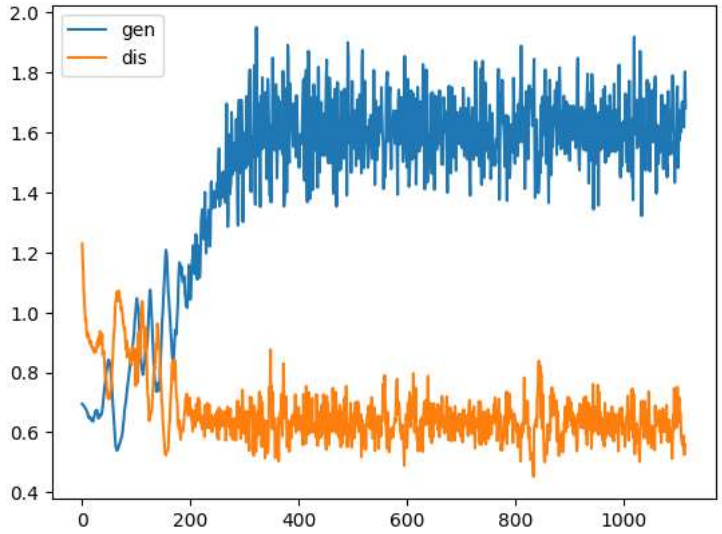
---------- epoch 19 ----------

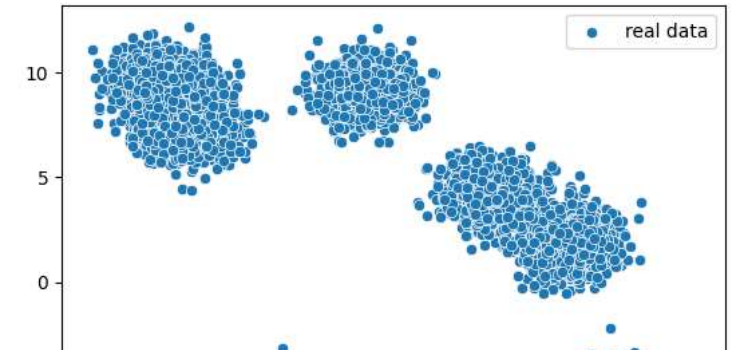

gen
dis
2.0
1.8
1.6
1.4
1.2
1.0
0.8
0.6
0
100
200
300
400
500
600
700
800

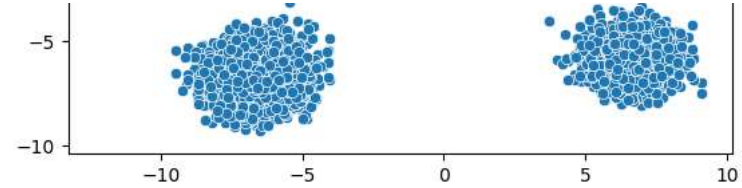




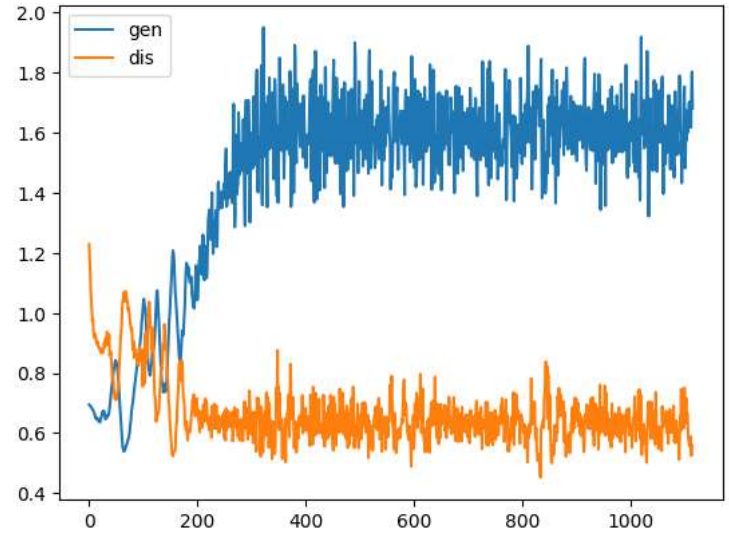
---------- epoch 29 ----------

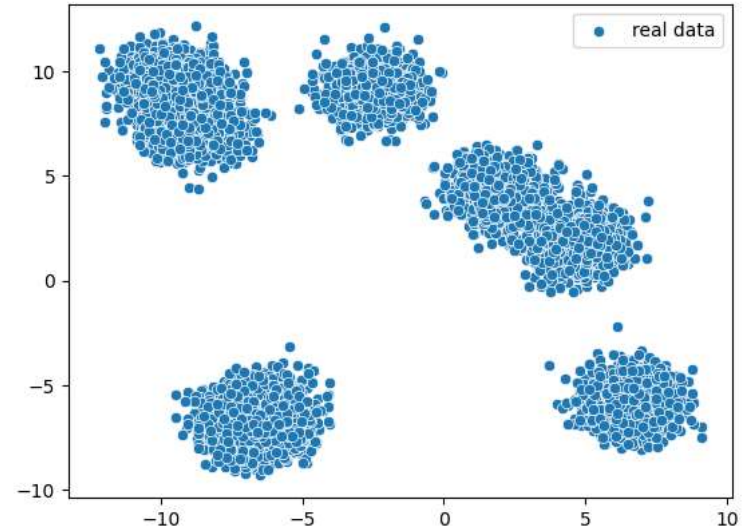






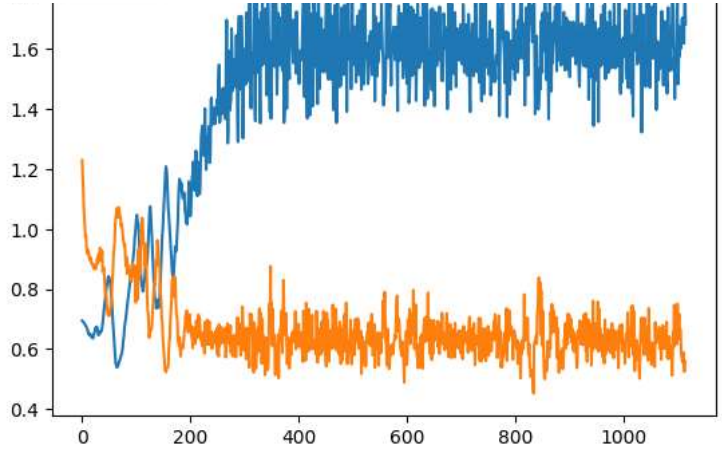
---------- epoch 39 ----------

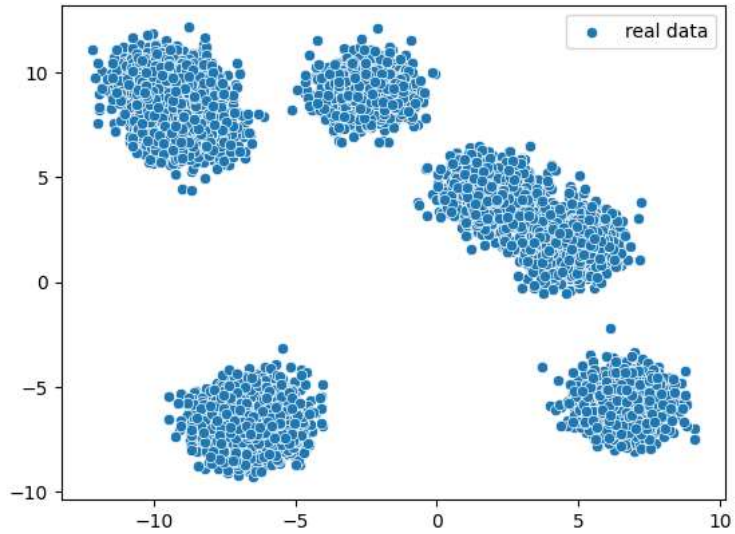




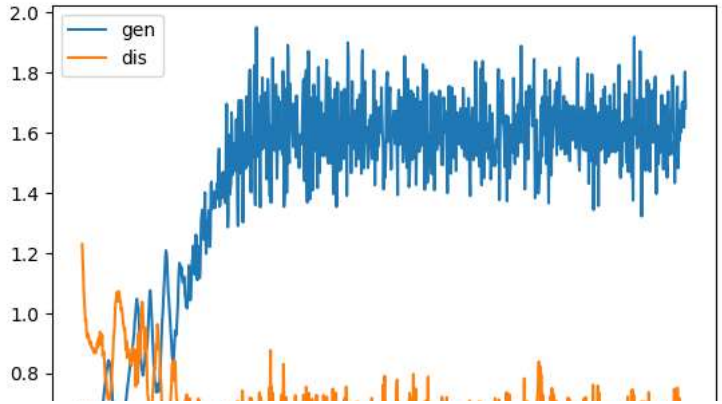
---------- epoch 49 ----------

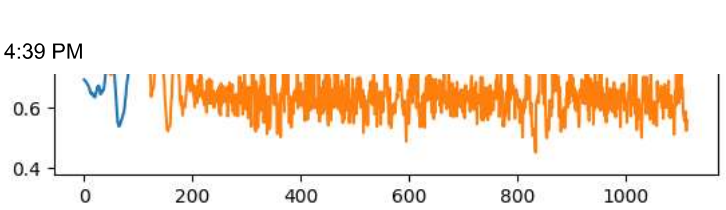


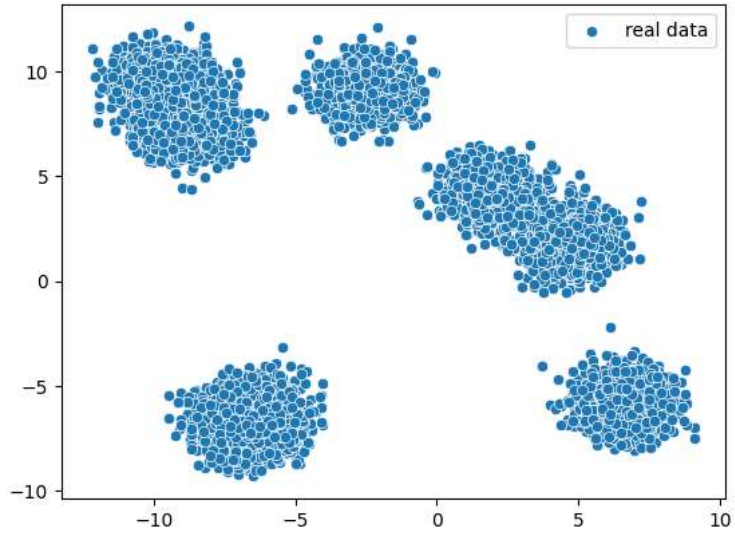

1.6
1.4
1.2
1.0
0.8
0.6
0.4
0
200
400
600
800
1000


real data
10
5
0
−5
−10
−10
−5
0
5
10

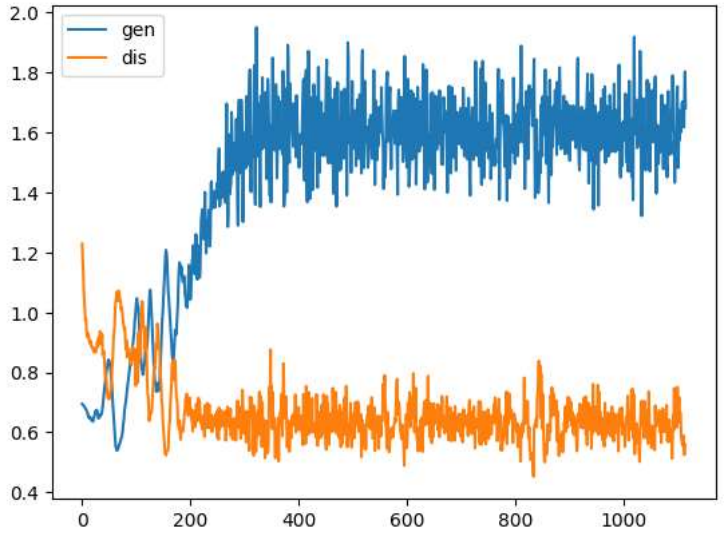
---------- epoch 59 ----------

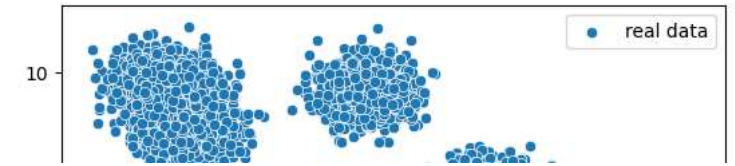

gen
dis
2.0
1.8
1.6
1.4
1.2
1.0
0.8

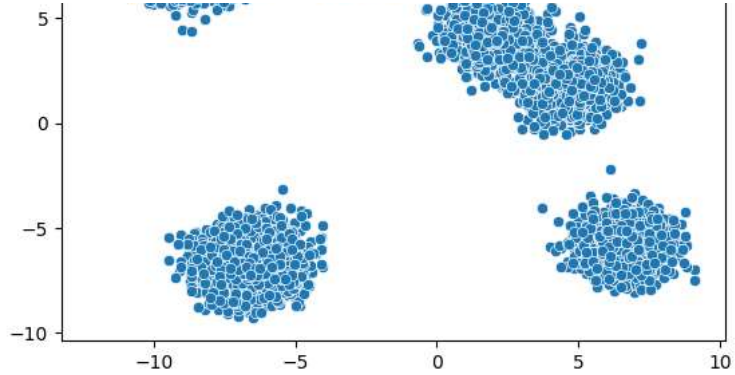

0.6
0.4
0
200
400
600
800
1000


real data
10
5
0
−5
−10
−10
−5
0
5
10

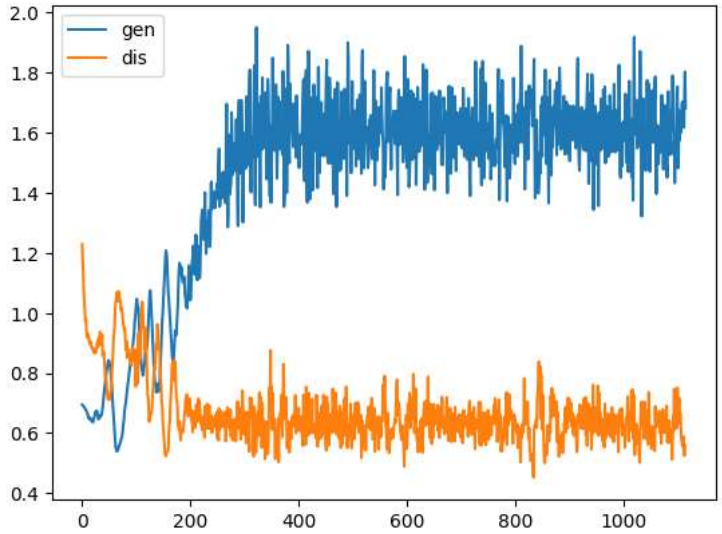
---------- epoch 69 ----------

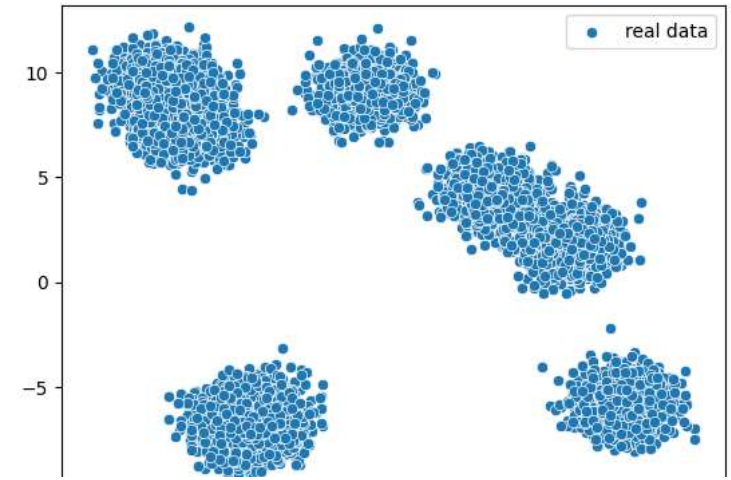

gen
dis
2.0
1.8
1.6
1.4
1.2
1.0
0.8
0.6
0.4
0
200
400
600
800
1000


real data
10


5
0
−5
−10
−10
−5
0
5
10

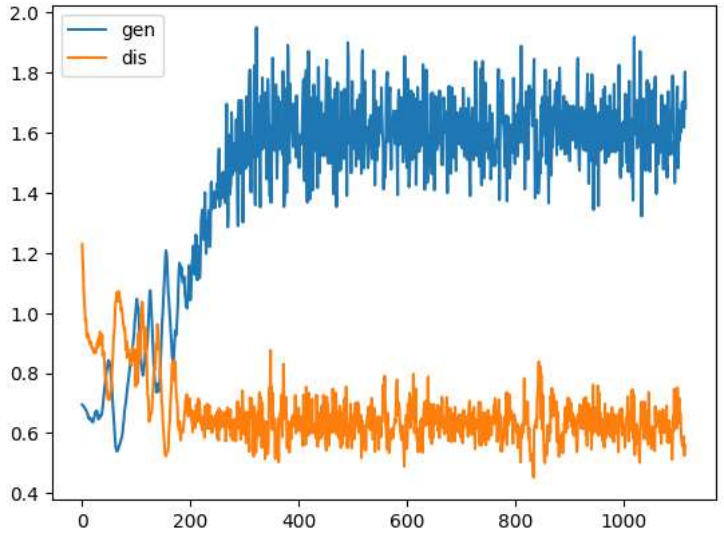
---------- epoch 79 ----------

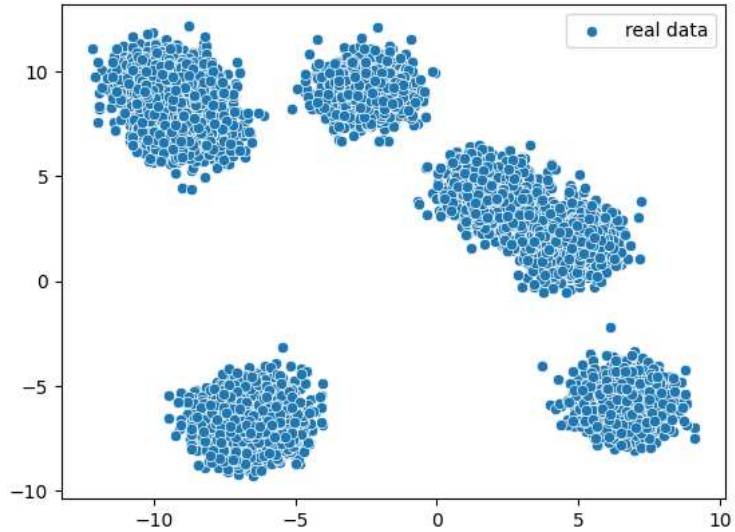

gen
dis
2.0
1.8
1.6
1.4
1.2
1.0
0.8
0.6
0.4
0
200
400
600
800
1000


real data
10
5
0
−5

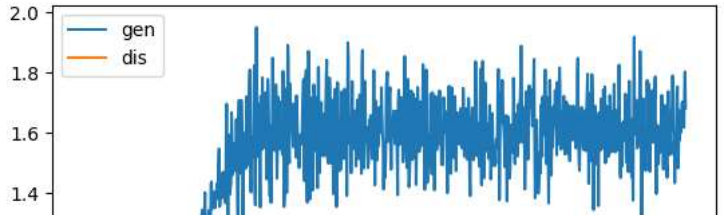


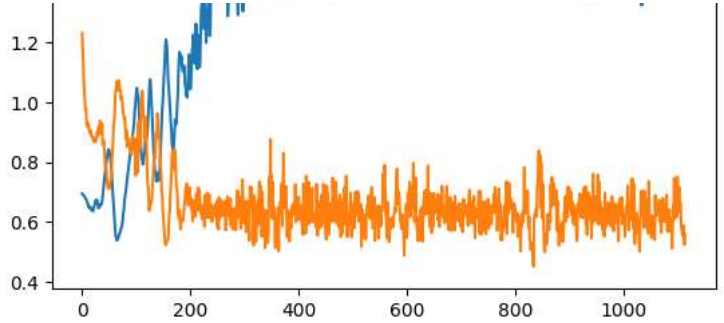

```
------------------------------ epoch 89 ------------------------------
```

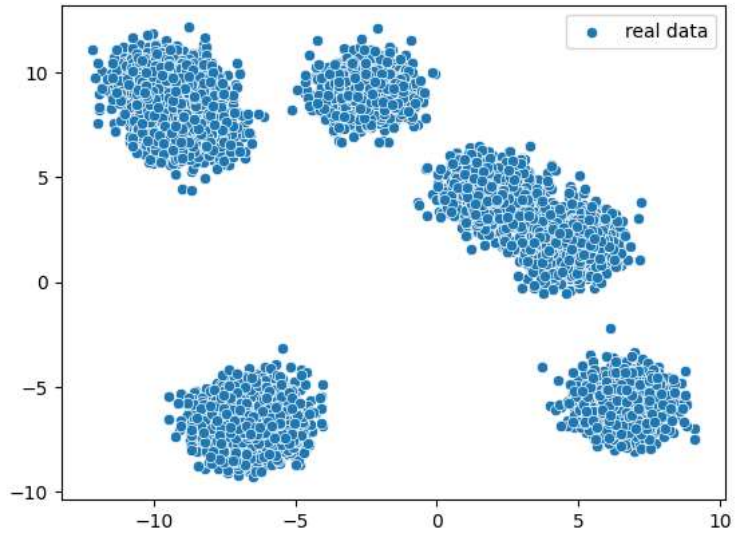




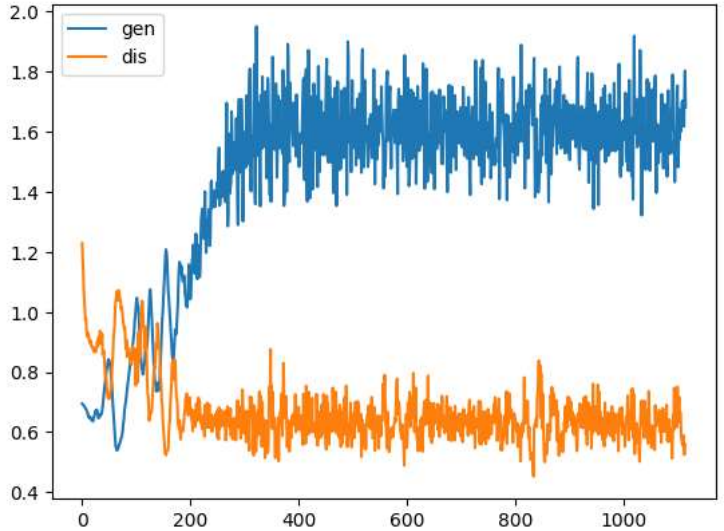

```
------------------------------ epoch 99 ------------------------------
```

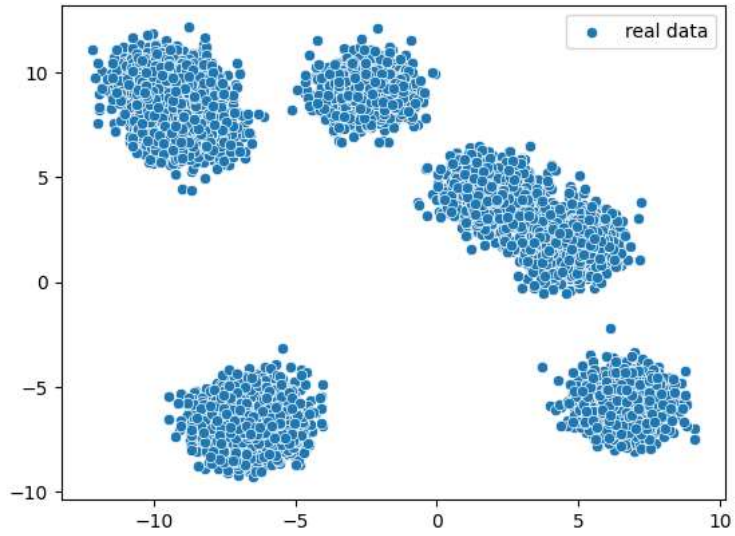



1.2
1.0
0.8
0.6
0.4
0
200
400
600
800
1000

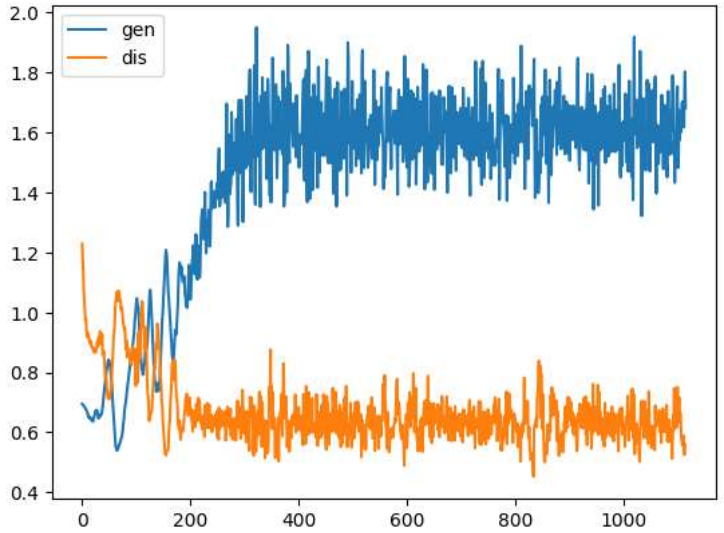

real data
10
5
0
−5
−10
−10
−5
0
5
10

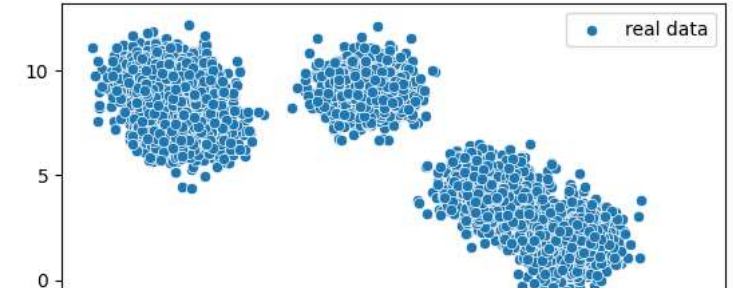
epoch 199

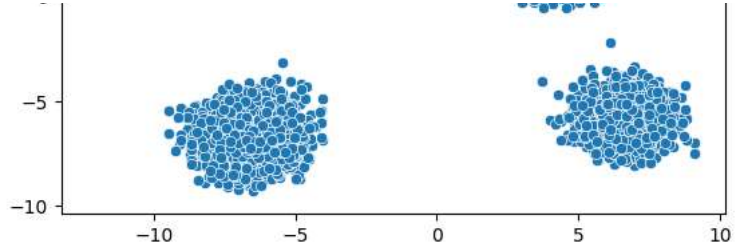

gen
dis
2.0
1.8
1.6
1.4
1.2
1.0
0.8
0.6
0.4
0
200
400
600
800
1000


real data
10
5
0
−5
−10
−10
−5
0
5
10

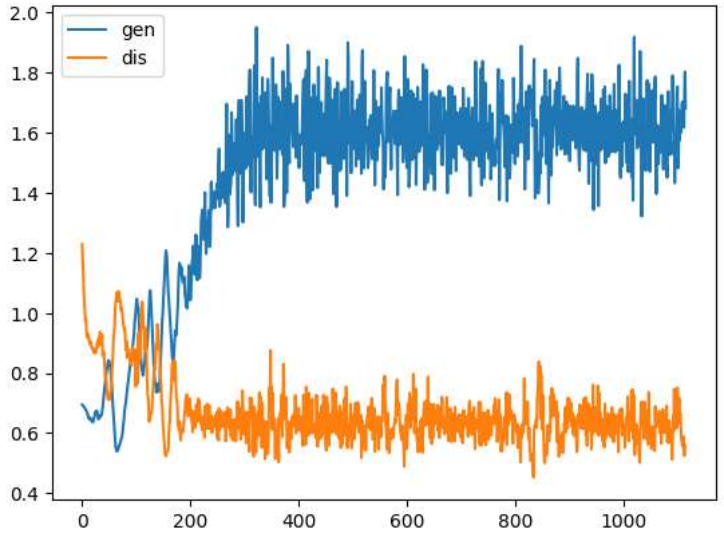
---------- epoch 299 ----------

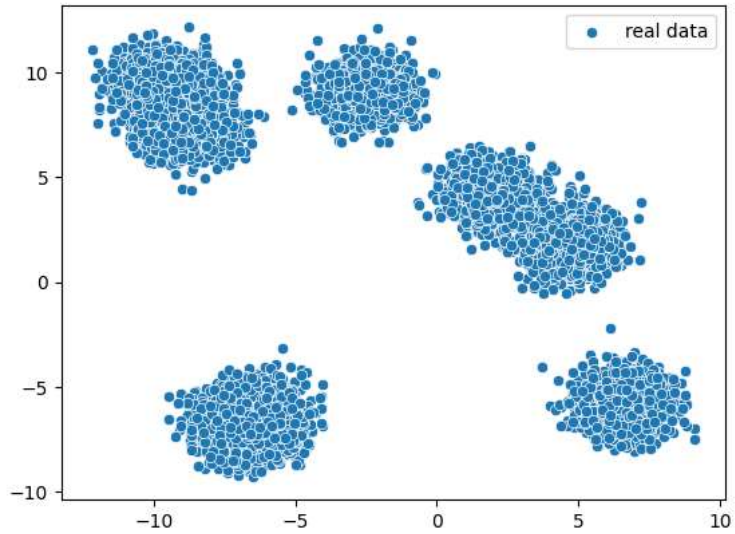

gen
dis
2.0
1.8
1.6
1.4
1.2
1.0
0.8
0.6
0.4
0
200
400
600
800
1000


real data
10
5
0



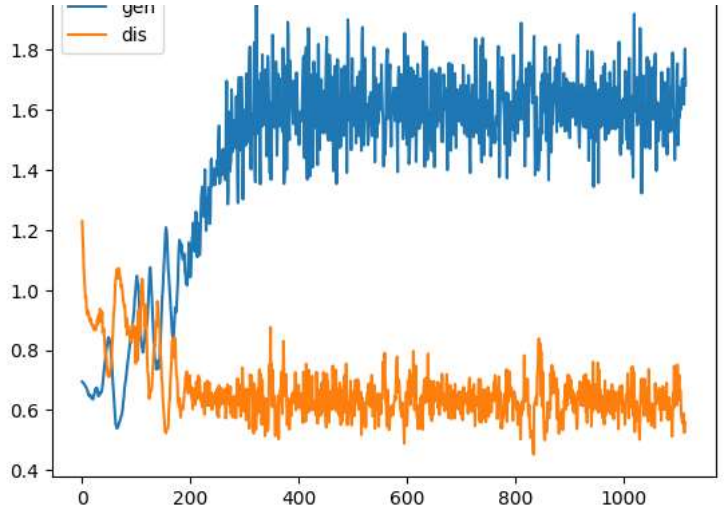
---------- epoch 399 ----------

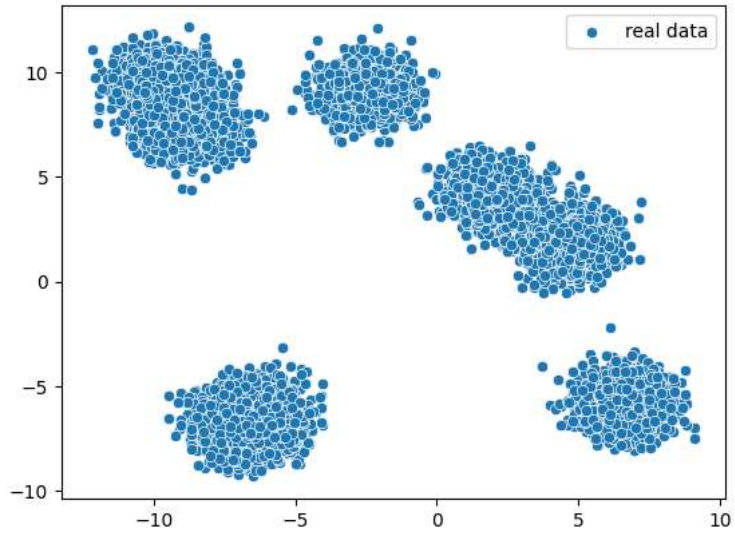




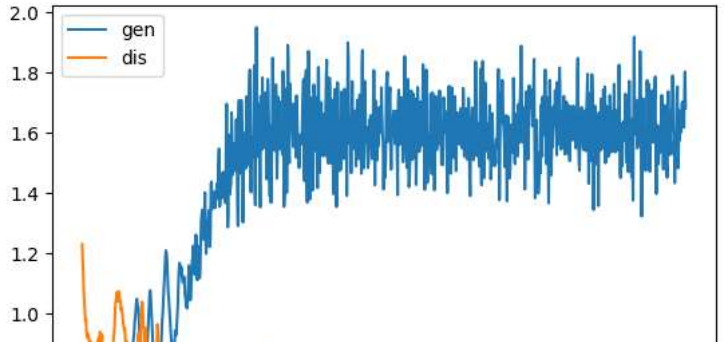
---------- epoch 499 ----------

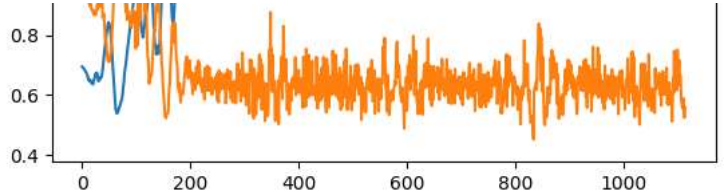


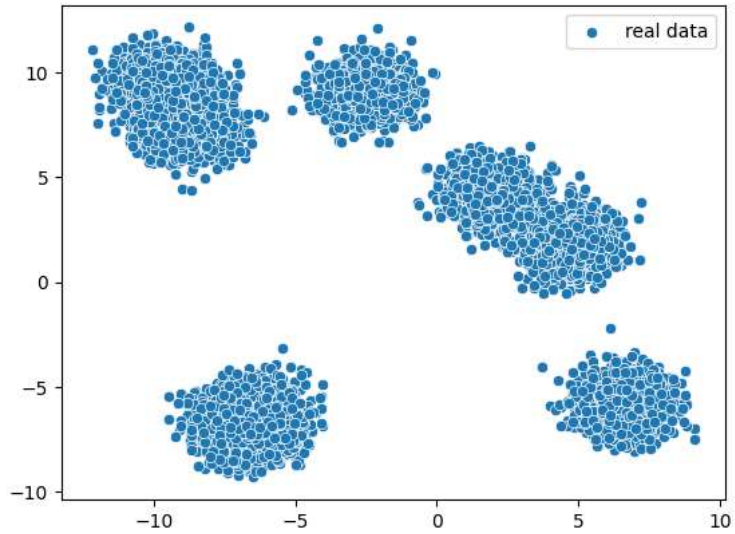

gen
dis
1.8
1.6
1.4
1.2
1.0
0.8
0.6
0.4
0
200
400
600
800
1000


real data
10
5
0
−5
−10
−10
−5
0
5
10

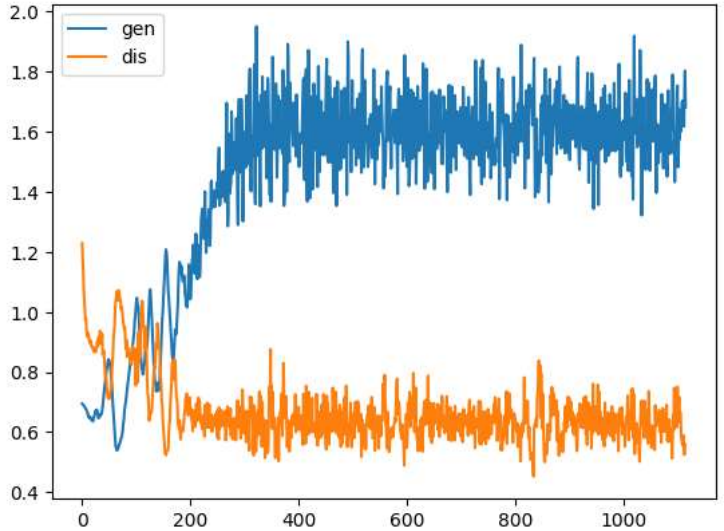
---------- epoch 599 ----------

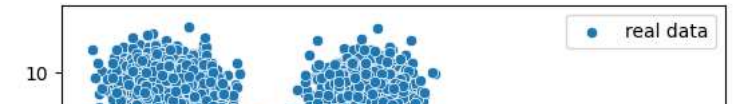

gen
dis
2.0
1.8
1.6
1.4
1.2
1.0

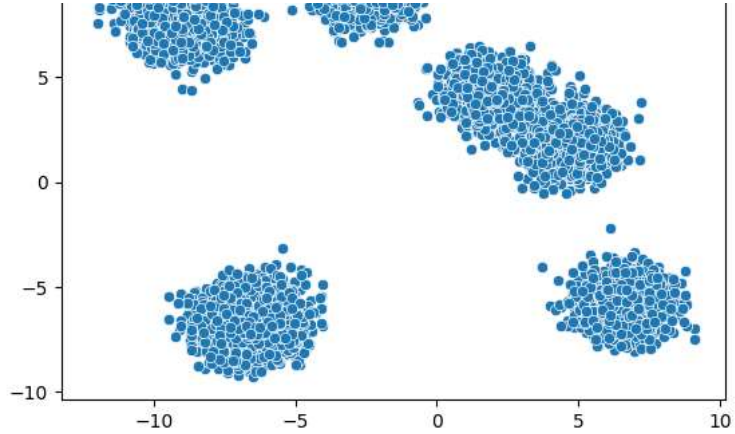

0.8
0.6
0.4
0
200
400
600
800
1000


real data
10
5
0
−5
−10
−10
−5
0
5
10

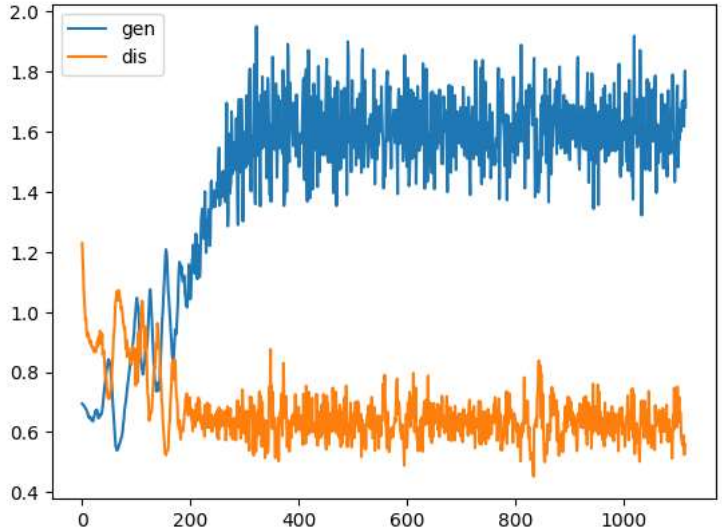
---------- epoch 699 ----------

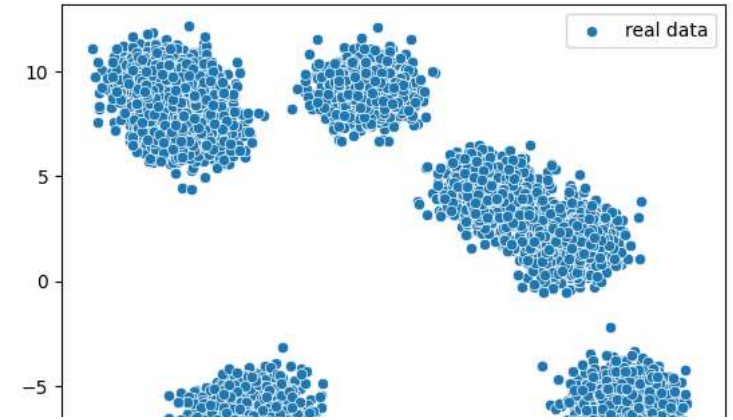

gen
dis
2.0
1.8
1.6
1.4
1.2
1.0
0.8
0.6
0.4
0
200
400
600
800
1000


real data
10


5
0
−5
−10
−10
−5
0
5
10

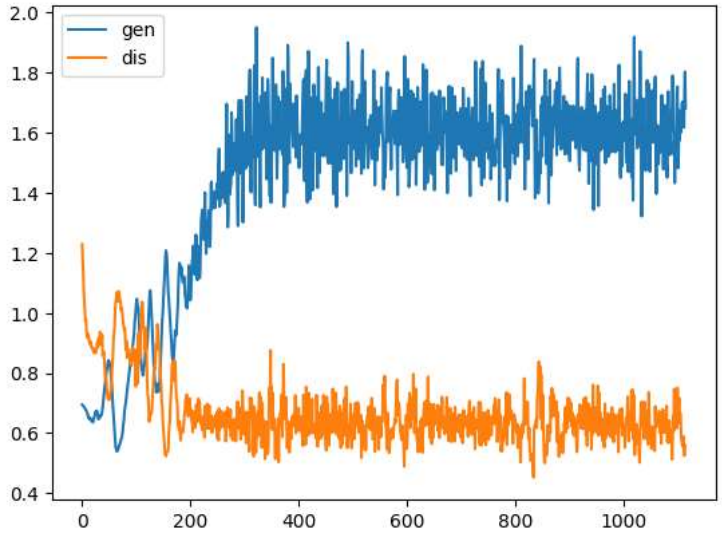
---------- epoch 799 ----------

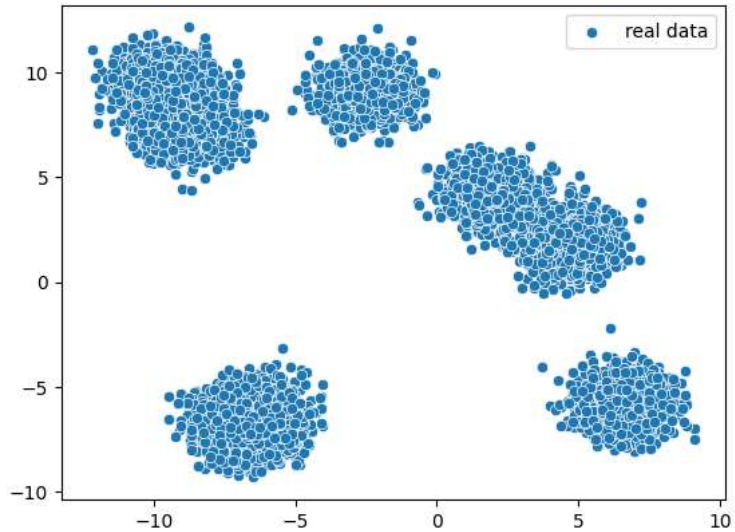

gen
dis
2.0
1.8
1.6
1.4
1.2
1.0
0.8
0.6
0.4
0
200
400
600
800
1000


real data
10
5
0
−5




```
------------------------------ epoch 899 ------------------------------
```





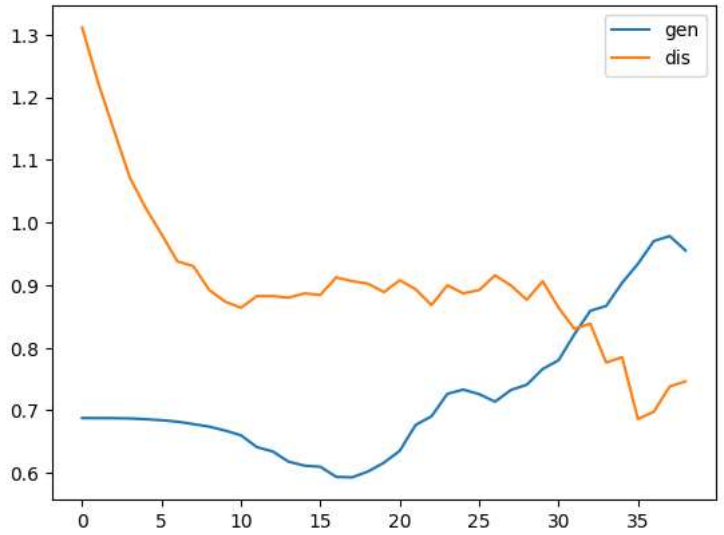

```
------------------------------ epoch 999 ------------------------------
```

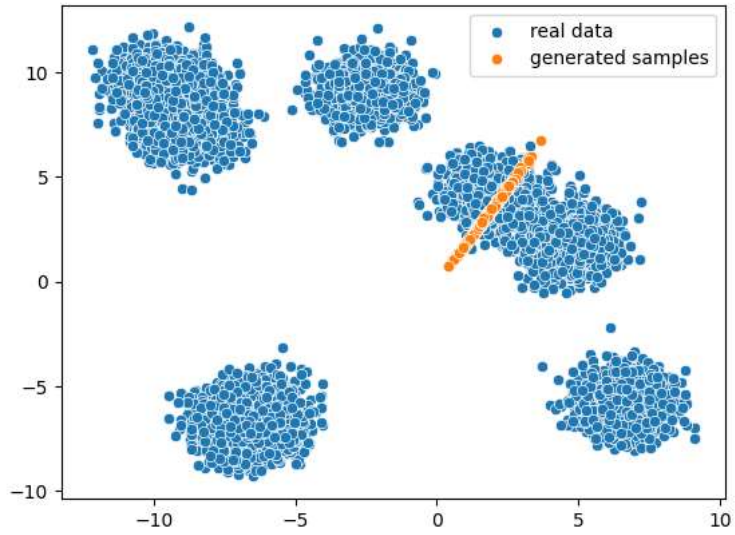



```
hparams = OmegaConf.create({
    'epochs': 1000,
```

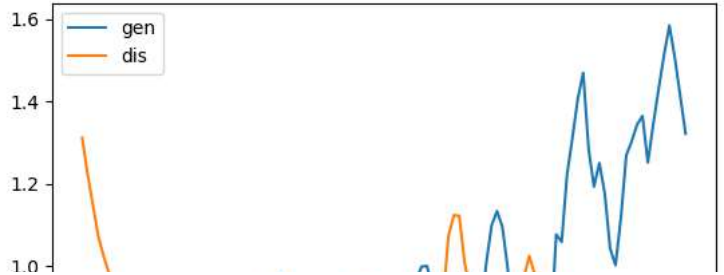
```
        'batch_size': 256,
        'gen_lr': 0.0001,
        'dis_lr': 0.0001,
        'beta1': 0.5,  # NOTE: important!
        'model_type': 'mlp',
        'generator': {
            'n_layers': 5,
            'hidden_dim': 512,
        },
        'discriminator': {
            'n_layers': 4,
            'hidden_dim': 512,
        },
        'input_noise_dim': 2,
    })
    create_and_train_gan(hparams, dataset_blobs(n_samples=10000, seed=seed), seed=seed)
```

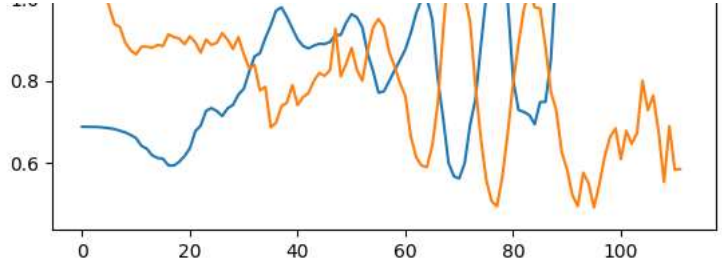
```
------------------------------ epoch 0 ------------------------------
```

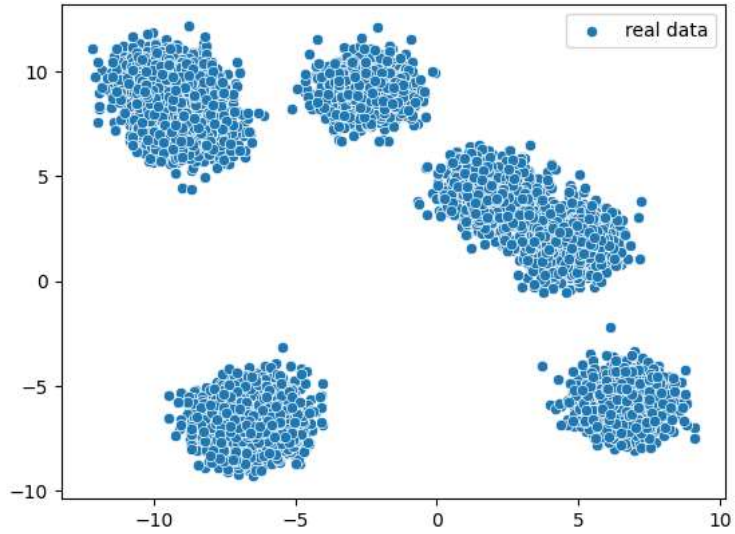




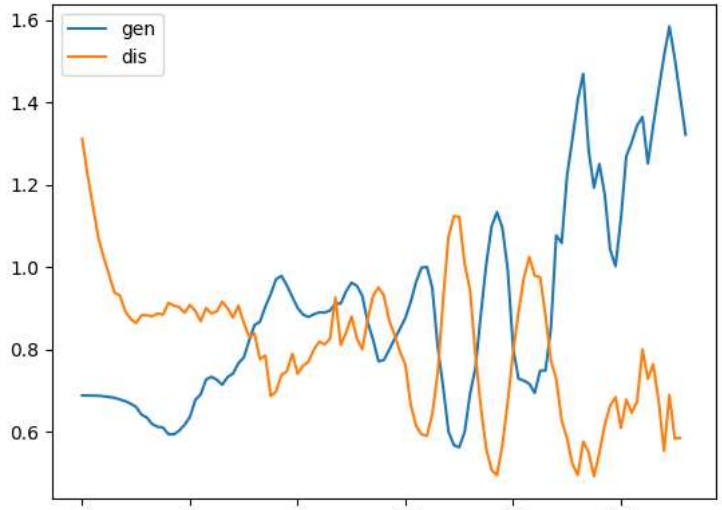

```
------------------------------ epoch 9 ------------------------------
```



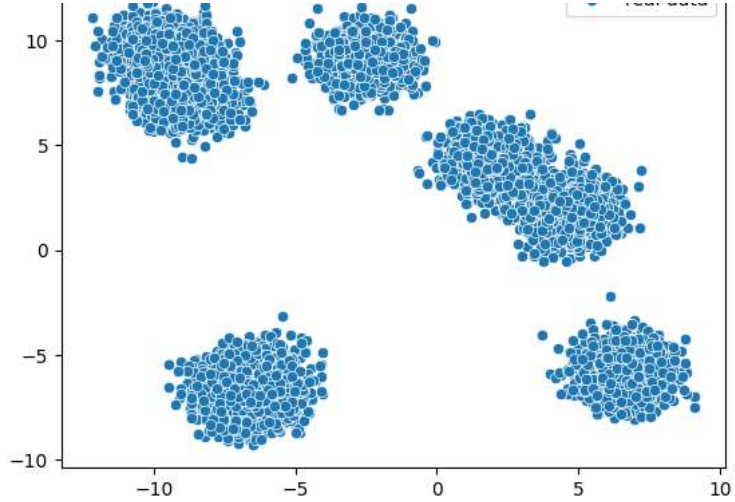

0.8
0.6
0
20
40
60
80
100


real data
10
5
0
−5
−10
−10
−5
0
5
10

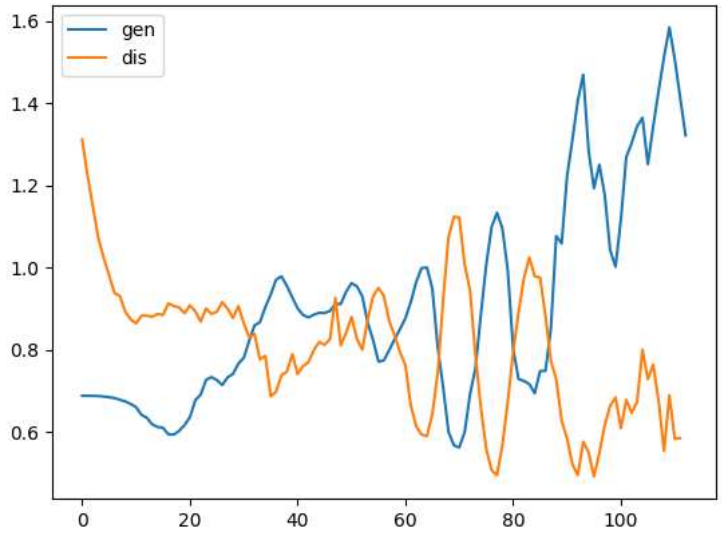
---------- epoch 19 ----------

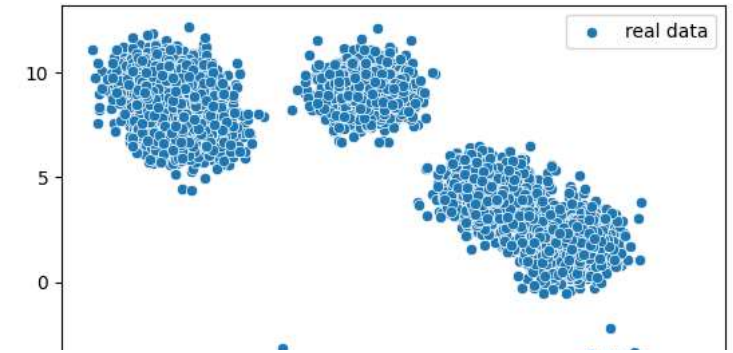

gen
dis
1.6
1.4
1.2
1.0
0.8
0.6
0
20
40
60
80
100

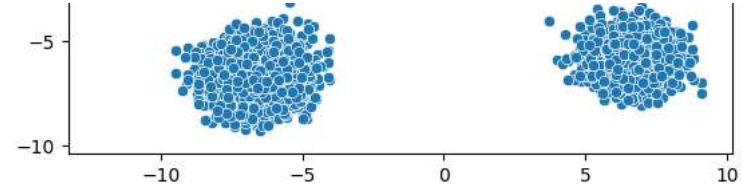

real data


10
5
0
−5
−10
−10
−5
0
5
10

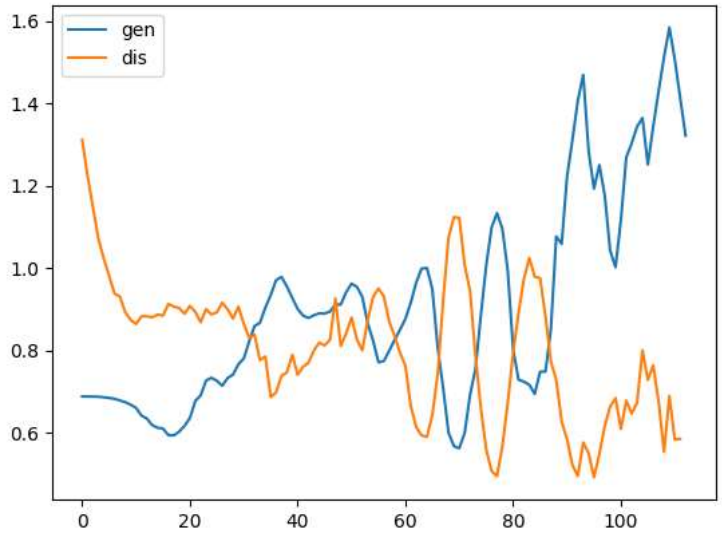
---------- epoch 29 ----------

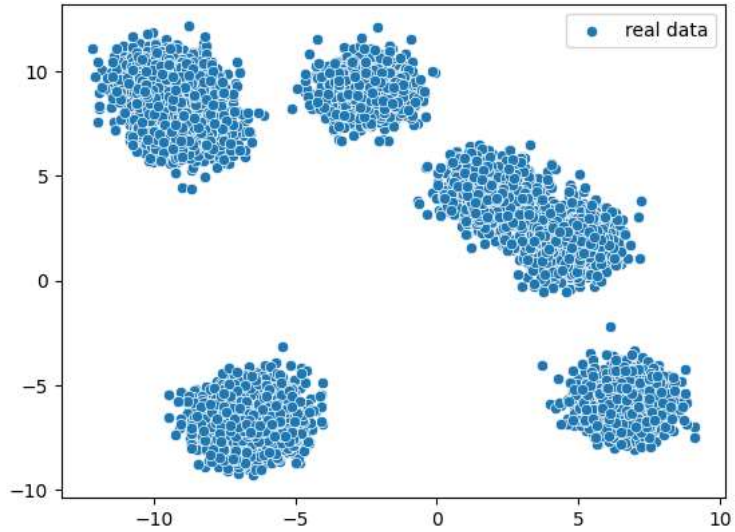

gen
dis
1.6
1.4
1.2
1.0
0.8
0.6
0
20
40
60
80
100


real data
10
5
0



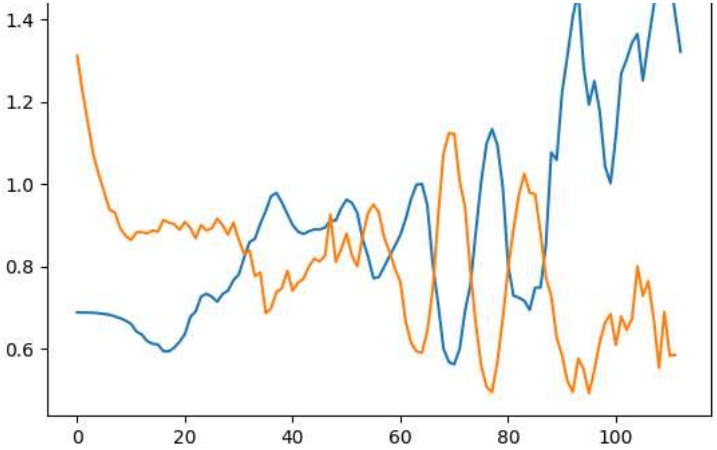
---------- epoch 39 ----------

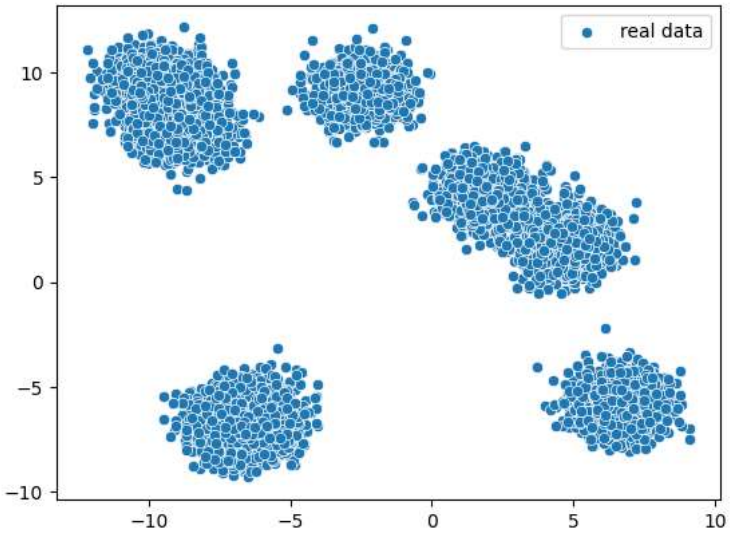




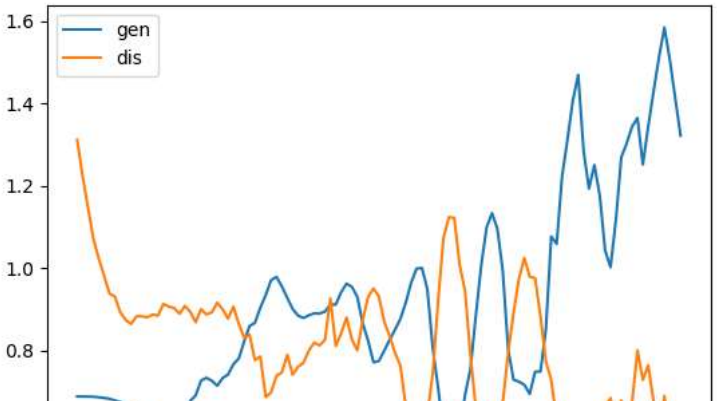
---------- epoch 49 ----------

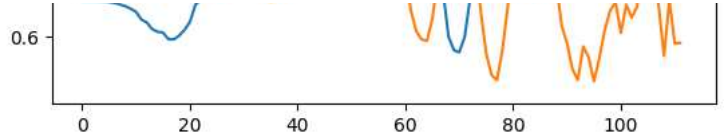


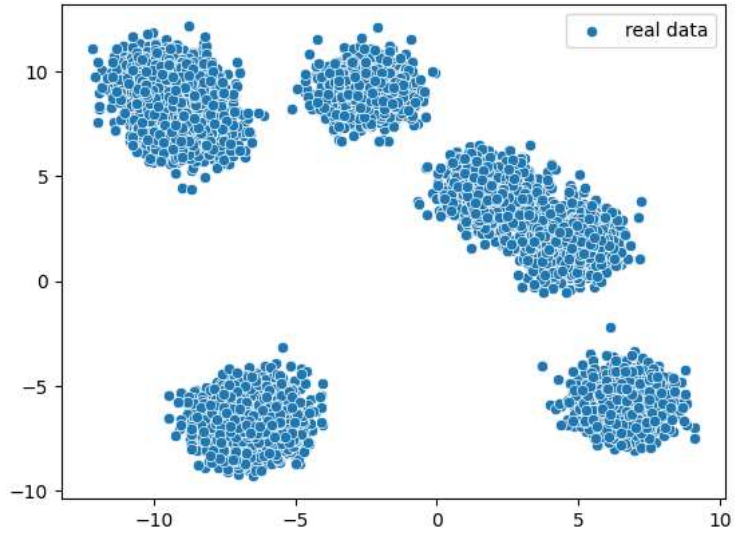

1.4
1.2
1.0
0.8
0.6
0
20
40
60
80
100


real data
10
5
0
−5
−10
−10
−5
0
5
10

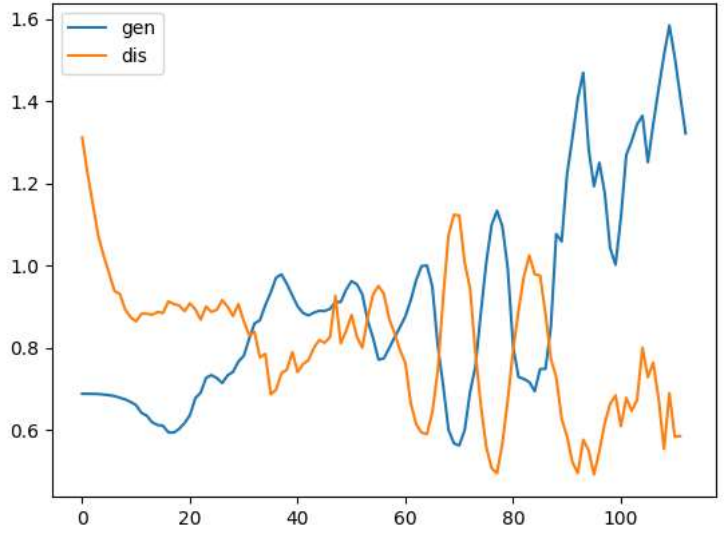
---------- epoch 59 ----------

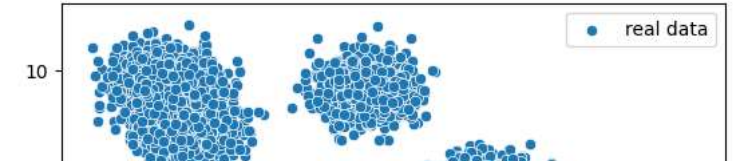

gen
dis
1.6
1.4
1.2
1.0
0.8

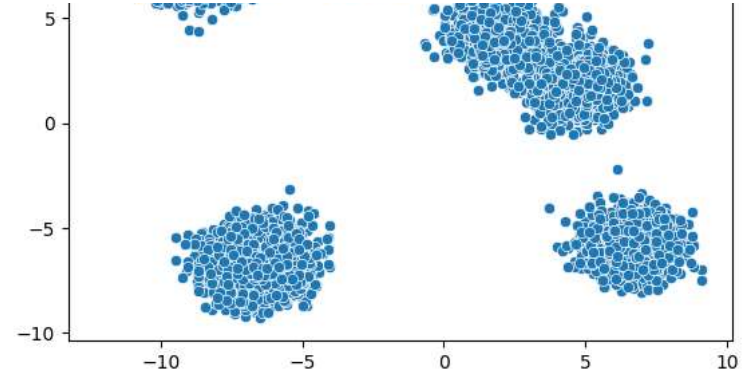

0.6
0
20
40
60
80
100


real data
10
5
0
−5
−10
−10
−5
0
5
10

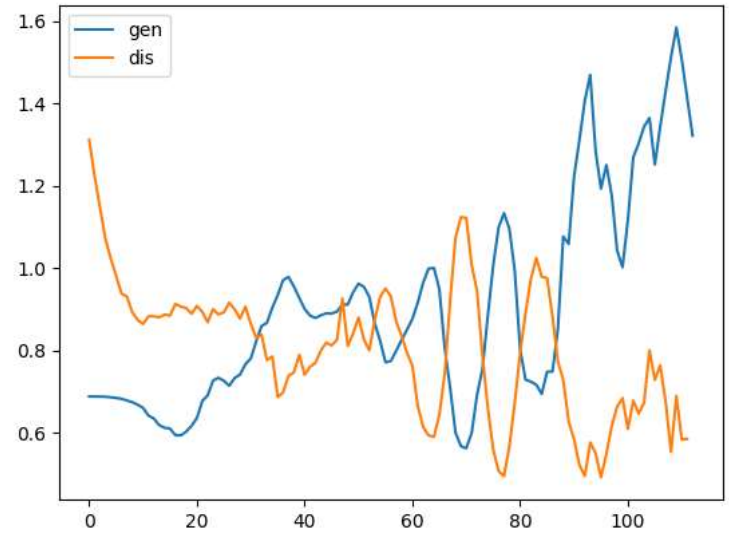
---------- epoch 69 ----------

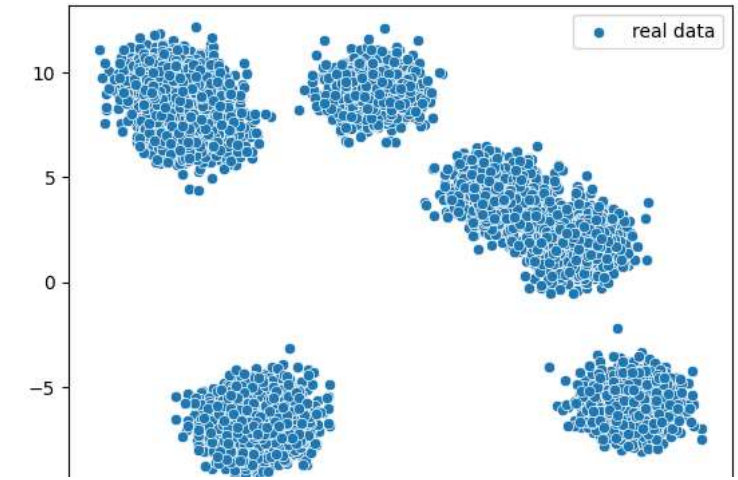

gen
dis
1.6
1.4
1.2
1.0
0.8
0.6
0
20
40
60
80
100


real data
10


5
0
−5
−10
−10
−5
0
5
10

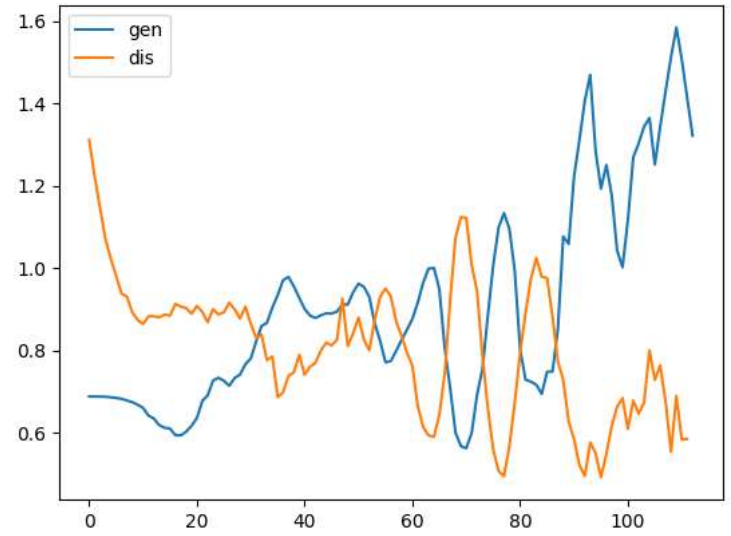
---------- epoch 79 ----------

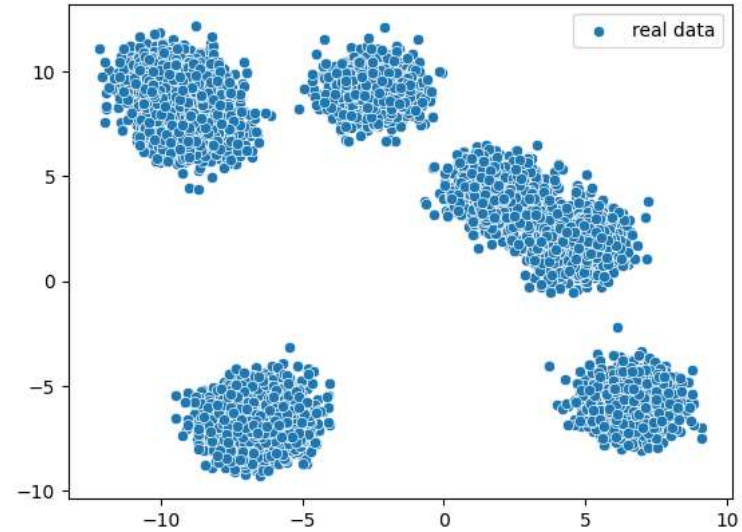

gen
dis
1.6
1.4
1.2
1.0
0.8
0.6
0
20
40
60
80
100


real data
10
5
0
−5



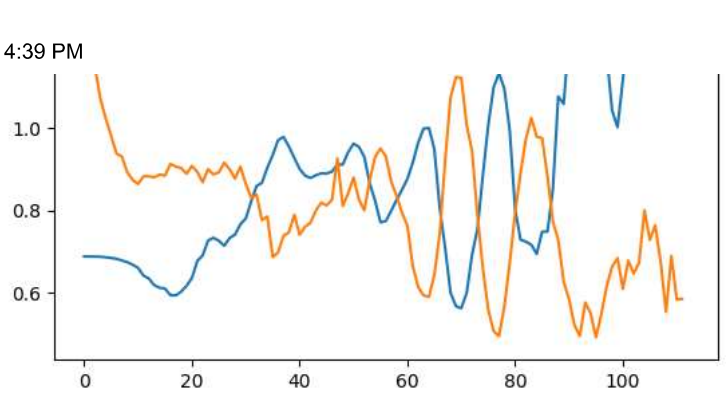
---------- epoch 89 ----------

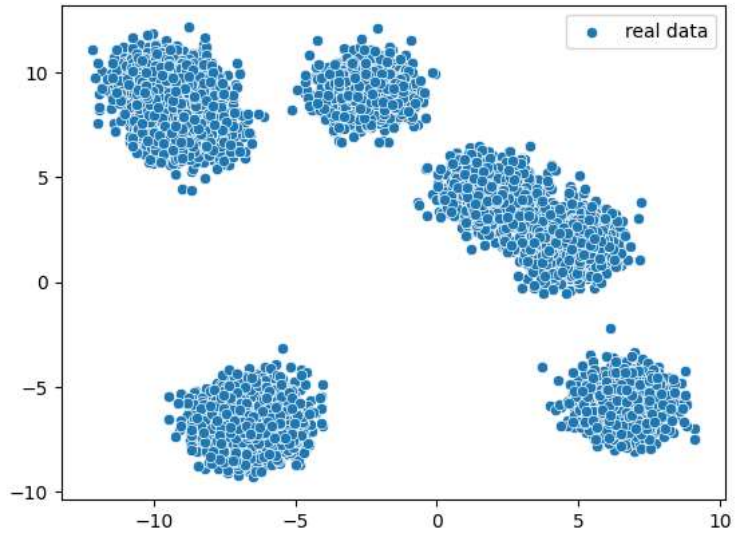




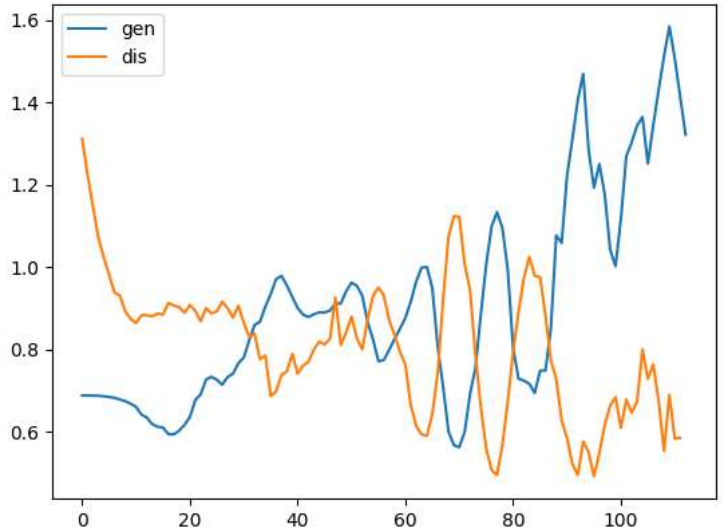
---------- epoch 99 ----------

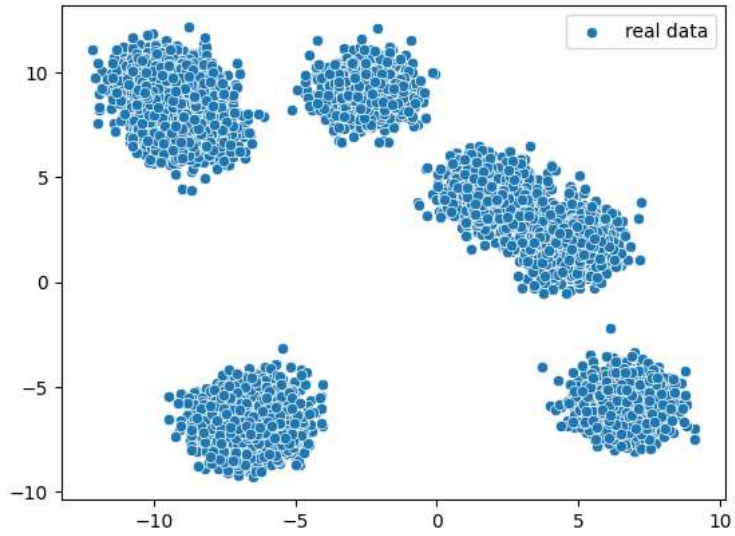



1.0
0.8
0.6
0
20
40
60
80
100

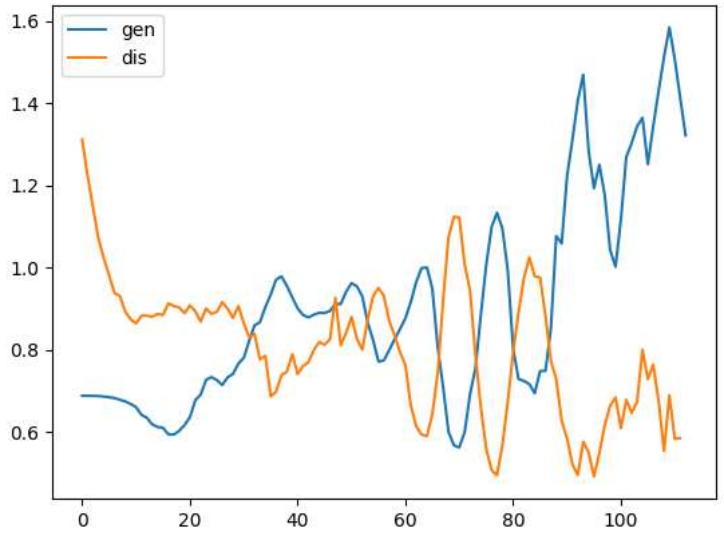

real data
10
5
0
−5
−10
−10
−5
0
5
10

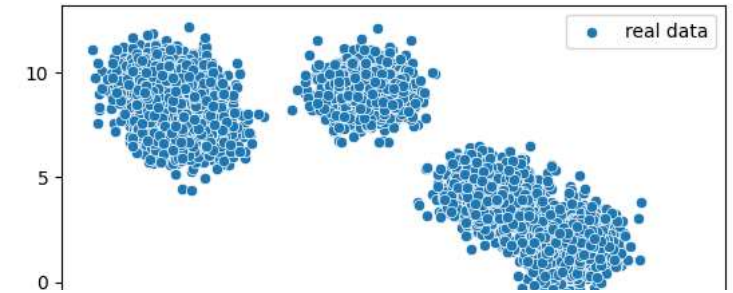
---------- epoch 199 ----------

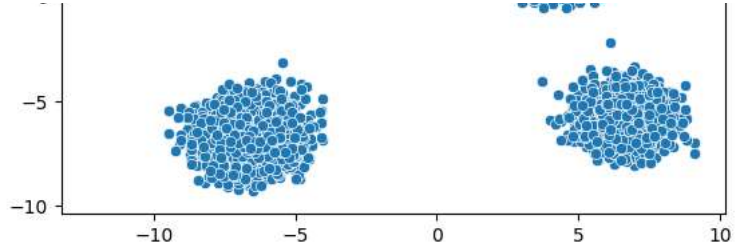

gen
dis
1.6
1.4
1.2
1.0
0.8
0.6
0
20
40
60
80
100


real data
10
5
0
−5
−10
−10
−5
0
5
10

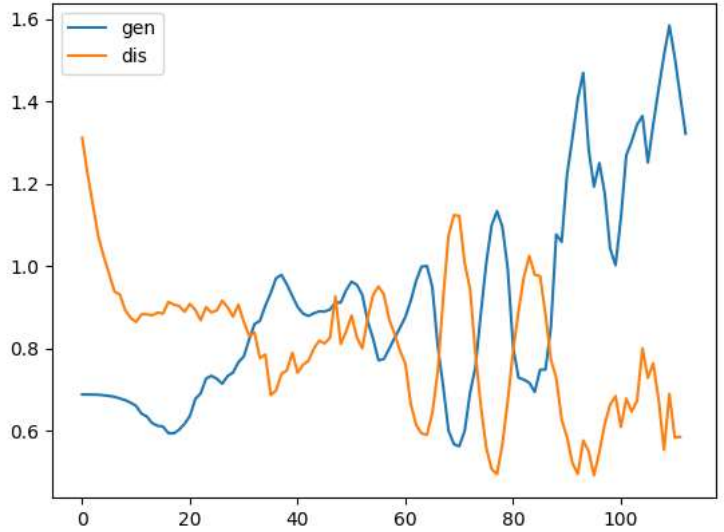
---------- epoch 299 ----------

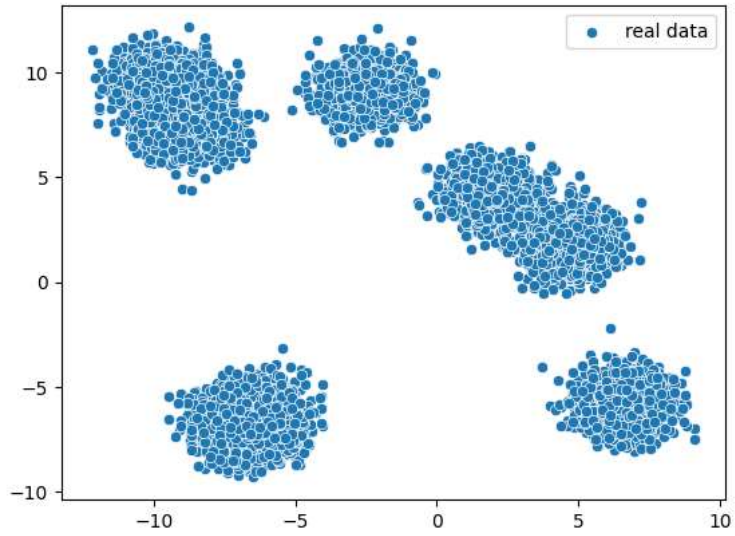

gen
dis
1.6
1.4
1.2
1.0
0.8
0.6
0
20
40
60
80
100


real data
10
5
0



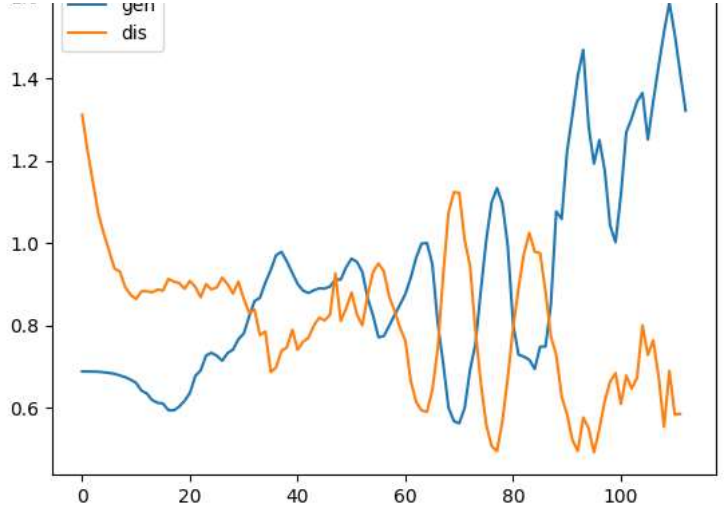
---------- epoch 399 ----------

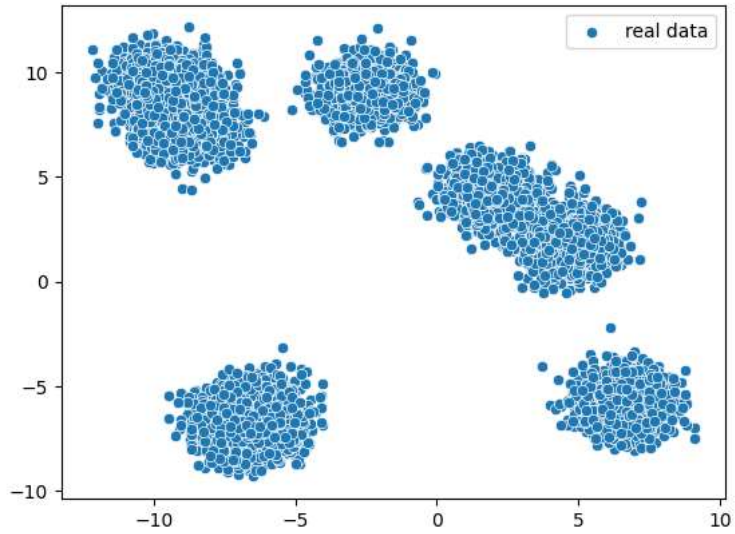




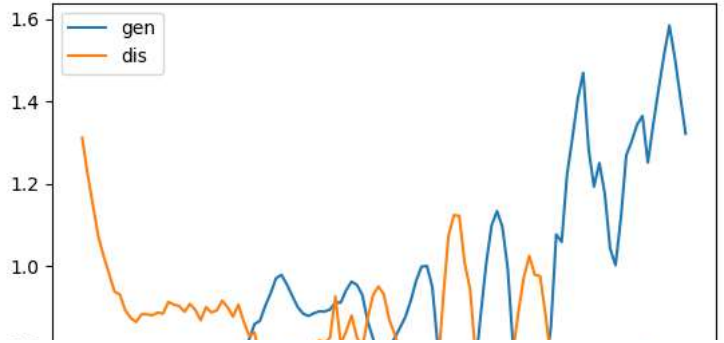
---------- epoch 499 ----------

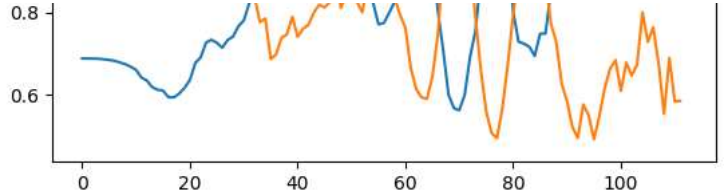


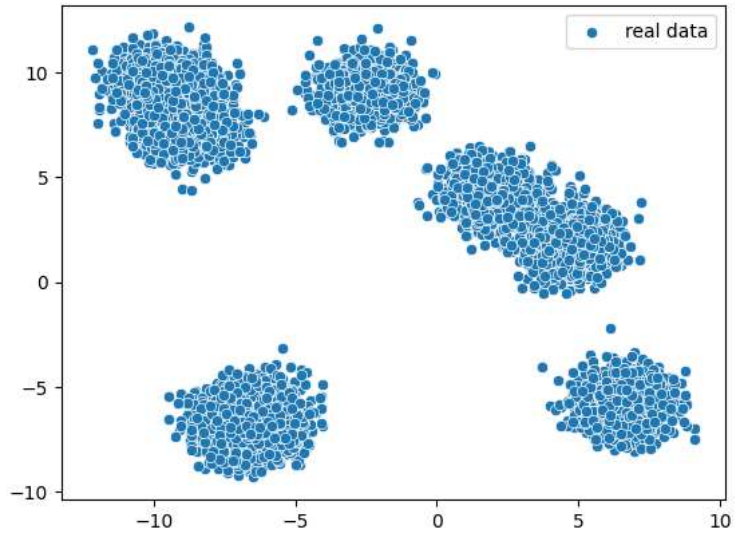

gen
dis
1.4
1.2
1.0
0.8
0.6
0
20
40
60
80
100


real data
10
5
0
−5
−10
−10
−5
0
5
10

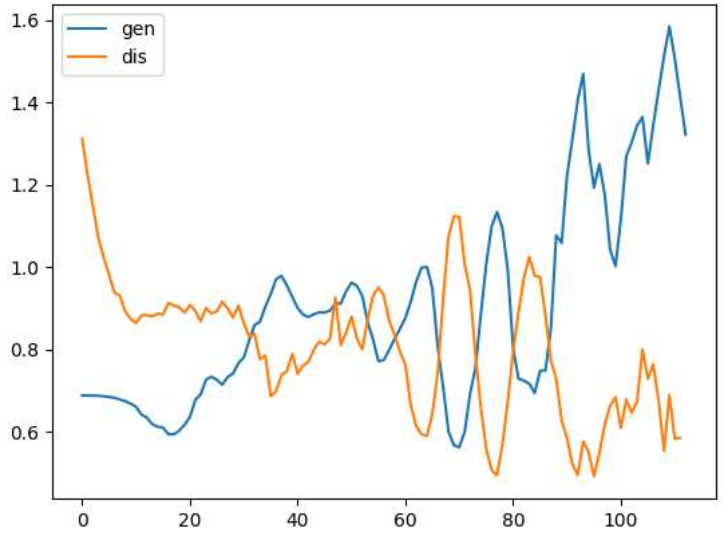
---------- epoch 599 ----------

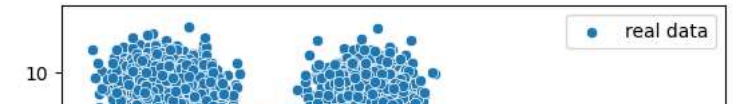

gen
dis
1.6
1.4
1.2
1.0

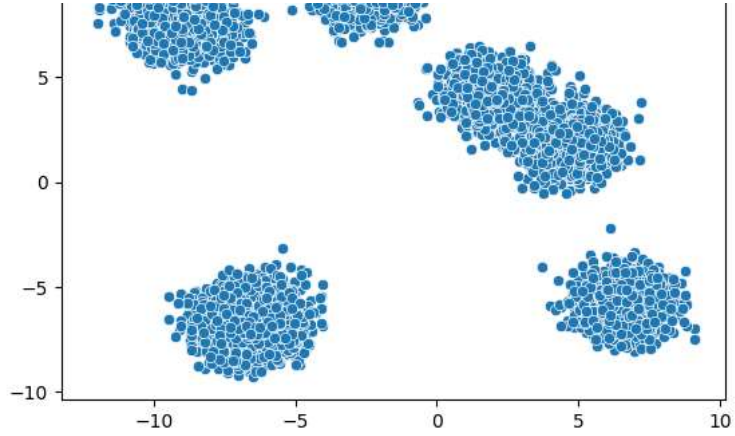

0.8
0.6
0
20
40
60
80
100


real data
10
5
0
−5
−10
−10
−5
0
5
10

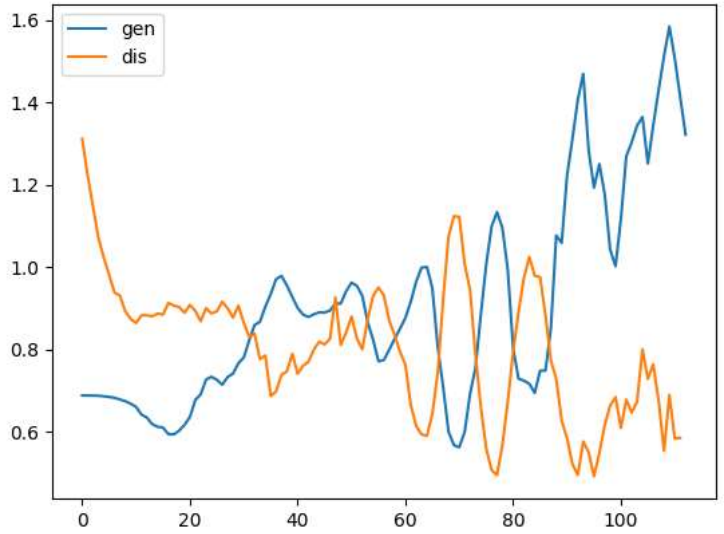
---------- epoch 699 ----------

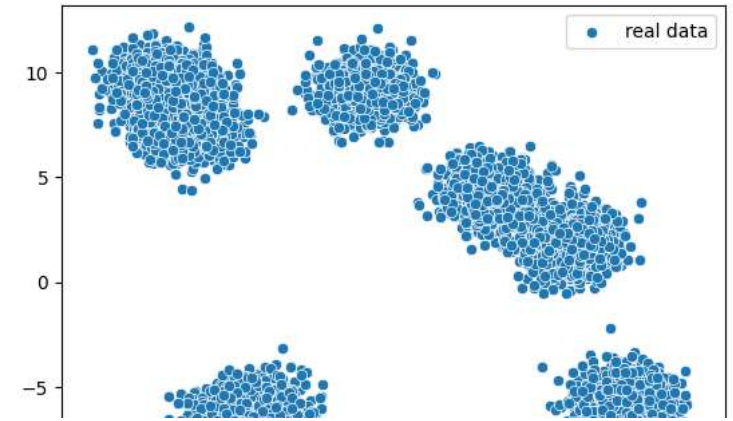

gen
dis
1.6
1.4
1.2
1.0
0.8
0.6
0
20
40
60
80
100


real data
10


5
0
−5
−10
−10
−5
0
5
10

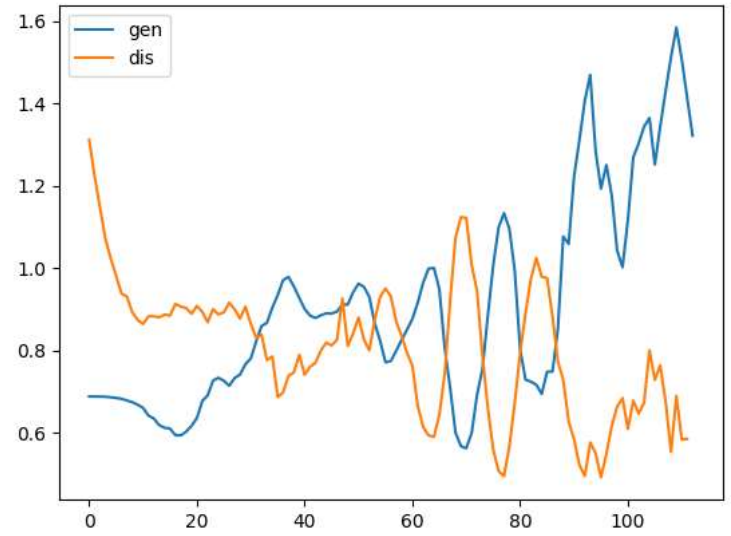
---------- epoch 799 ----------

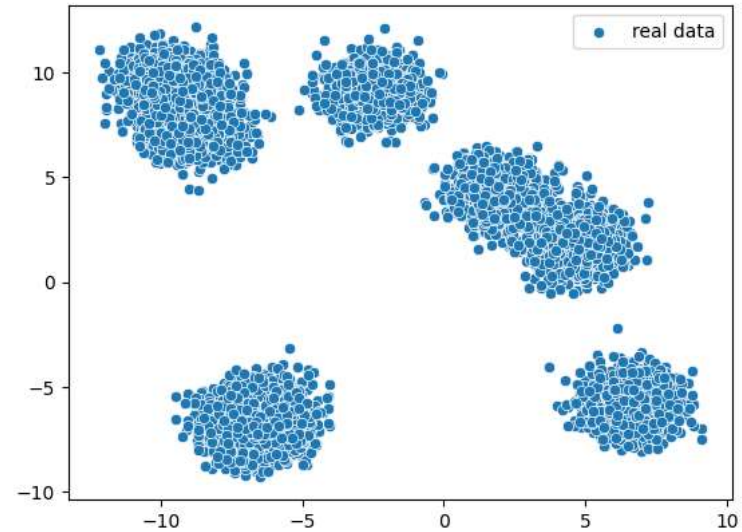

gen
dis
1.6
1.4
1.2
1.0
0.8
0.6
0
20
40
60
80
100


real data
10
5
0
−5



epoch 899





epoch 999



If you get a similar result feel free to play with the hyperparameter values! Which of them seem important? Which of them the model is not so sensitive?

You can also try to change the random seed and see if the results change significantly. Do the results seem more consistent across different seeds for different hyperparamer settings?

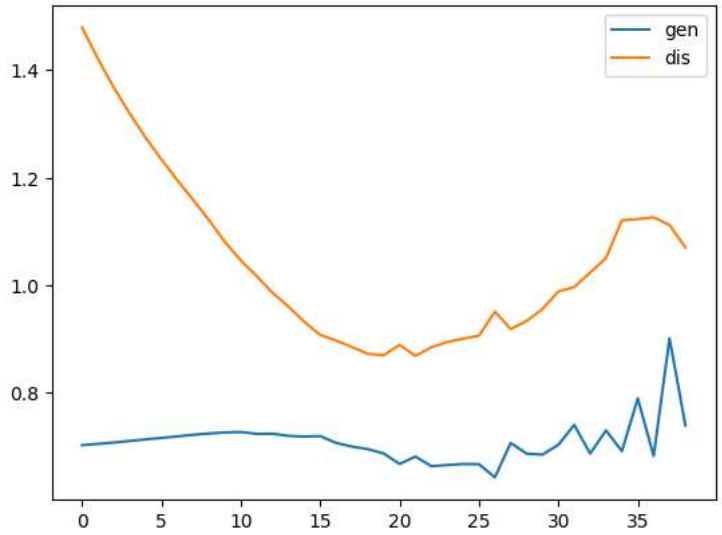
(Task) Train the GAN model: Swiss roll data

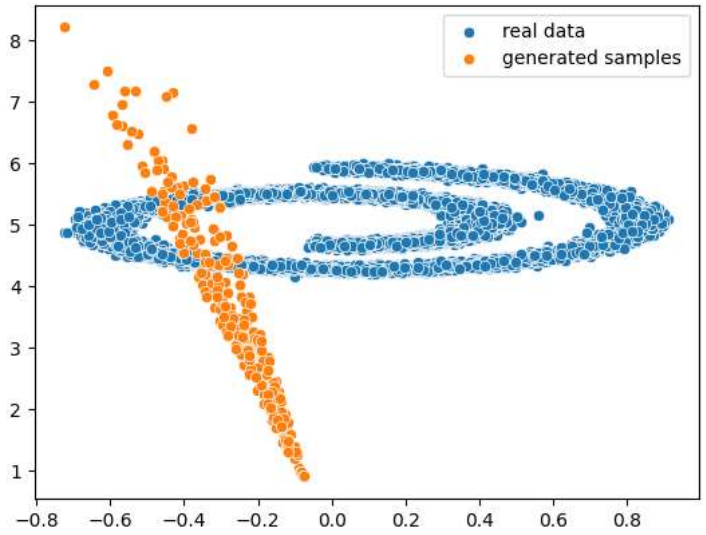
Now, let's try a slightly difficult problem!

```
hparams = OmegaConf.create({
    'epochs': 1000,
    'batch_size': 256,
    'gen_lr': 0.0001,
    'dis_lr': 0.0001,
    'beta1': 0.5,  # NOTE: important!
    'model_type': 'mlp',
    'generator': {
        'n_layers': 3,
        'hidden_dim': 512,
    },
    'discriminator': {
        'n_layers': 5,
        'hidden_dim': 512,
    },
    'input_noise_dim': 2,
})

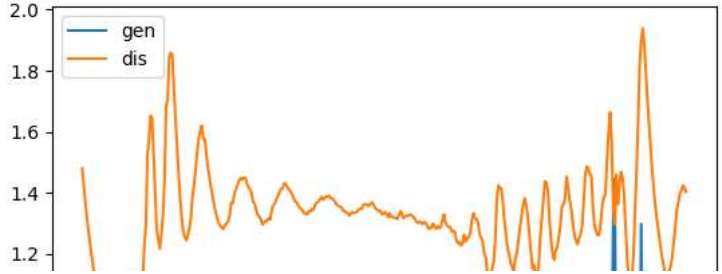
create_and_train_gan(hparams, dataset_swiss_roll(n_samples=10000, seed=seed), seed=seed)
```

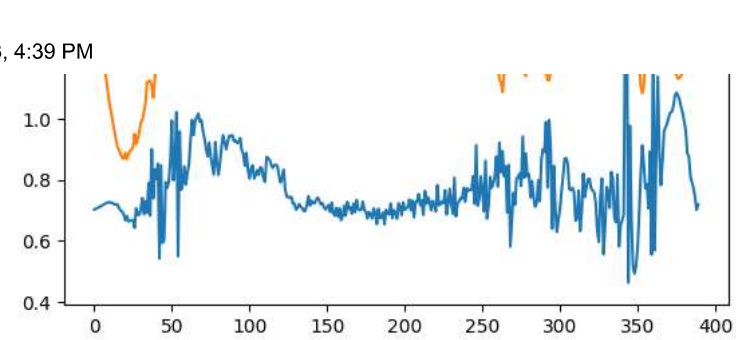
```
------------------------------ epoch 0 ------------------------------
```

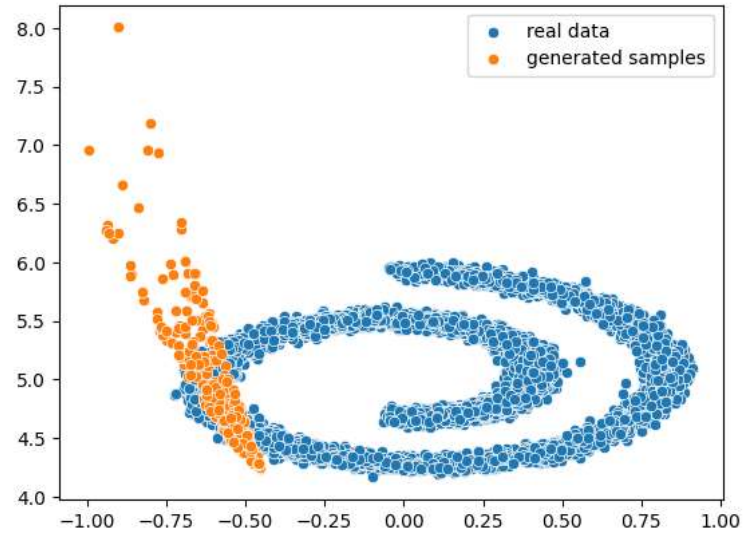




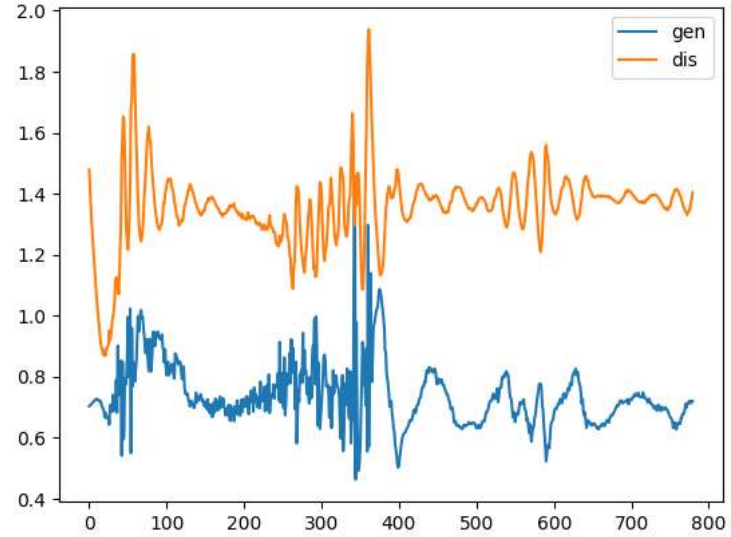

```
------------------------------ epoch 9 ------------------------------
```

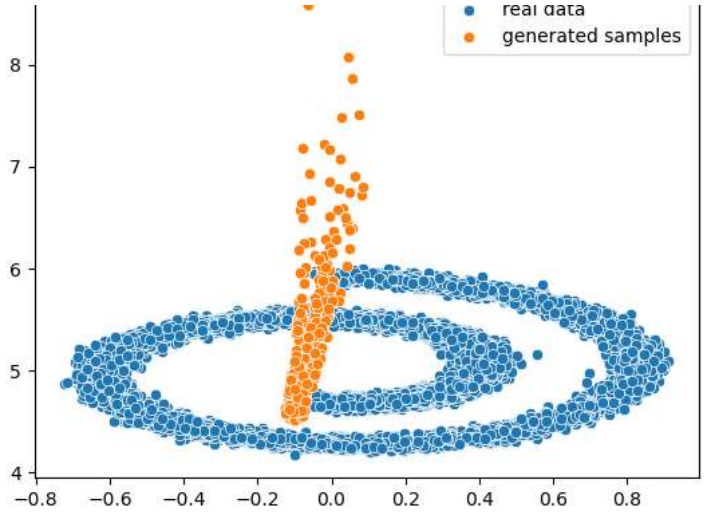



1.0
0.8
0.6
0.4
0
50
100
150
200
250
300
350
400

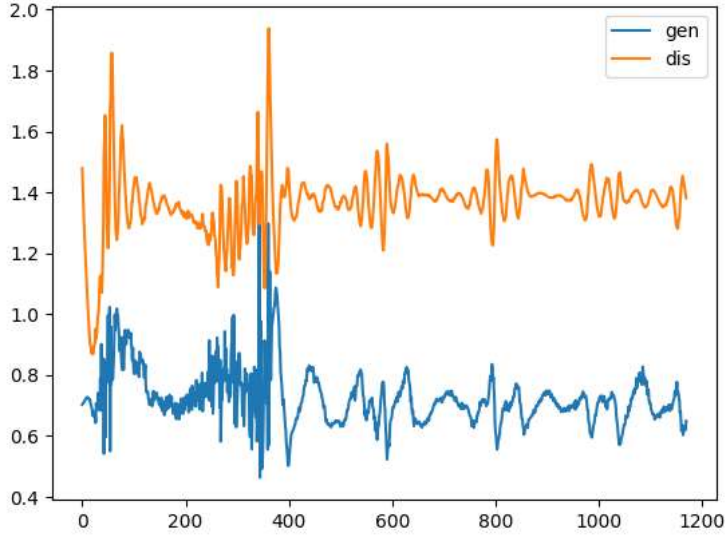

real data
generated samples
8.0
7.5
7.0
6.5
6.0
5.5
5.0
4.5
4.0
−1.00
−0.75
−0.50
−0.25
0.00
0.25
0.50
0.75
1.00

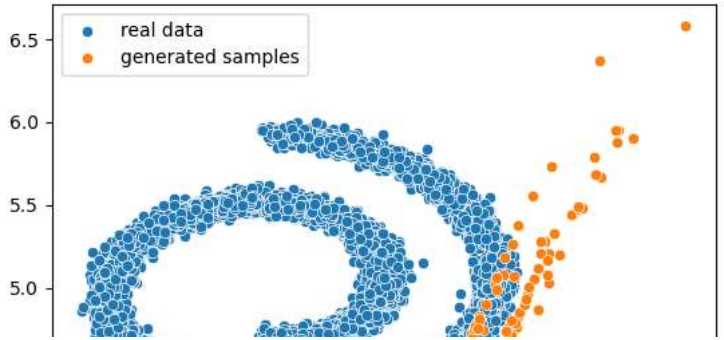
---------- epoch 19 ----------

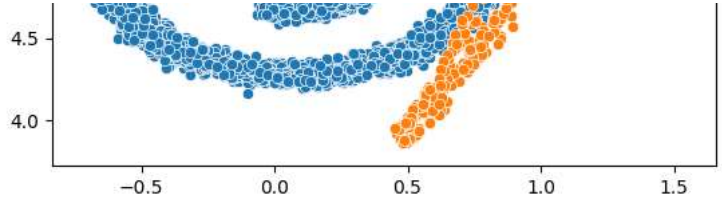

gen
dis
2.0
1.8
1.6
1.4
1.2
1.0
0.8
0.6
0.4
0
100
200
300
400
500
600
700
800


real data
generated samples

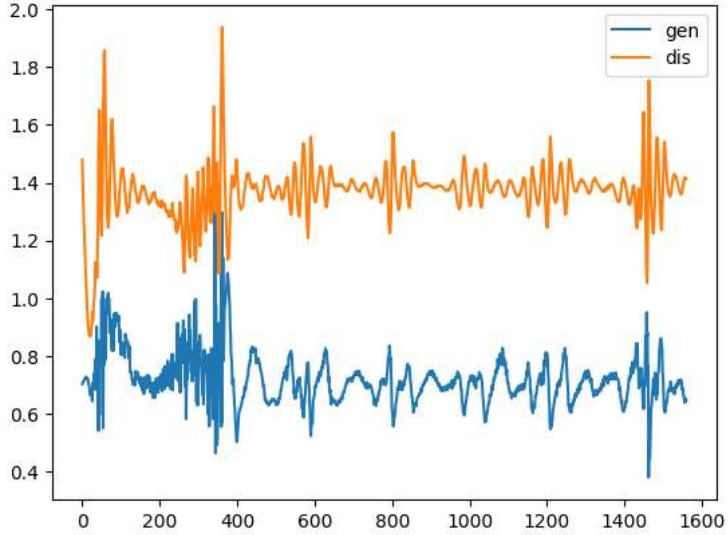
---------- epoch 29 ----------

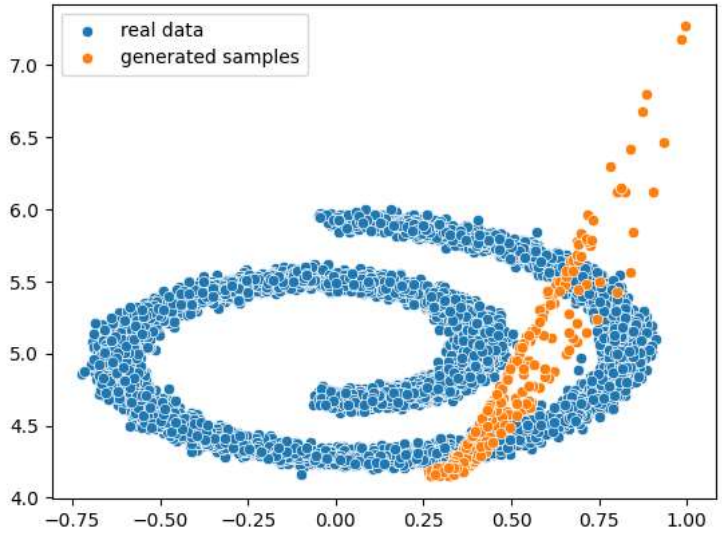

gen
dis


real data
generated samples

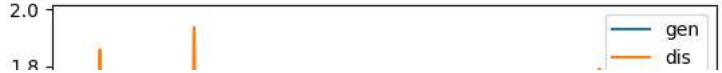


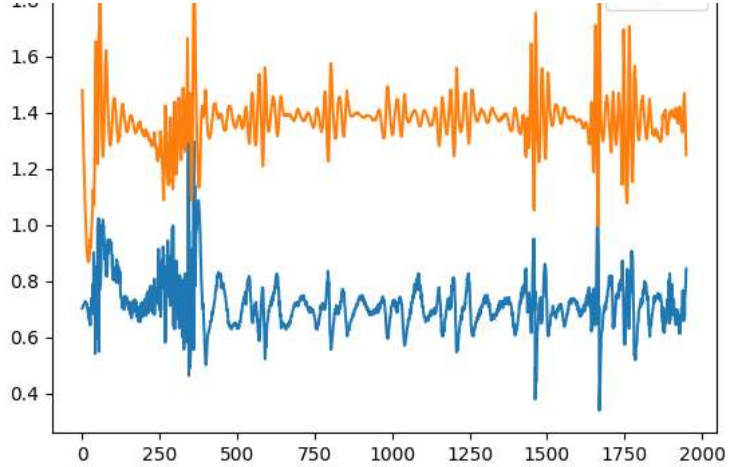
---------- epoch 39 ----------

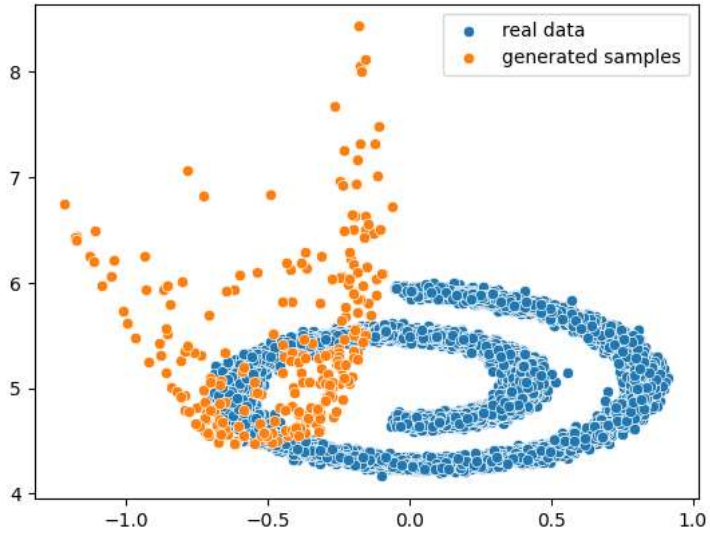




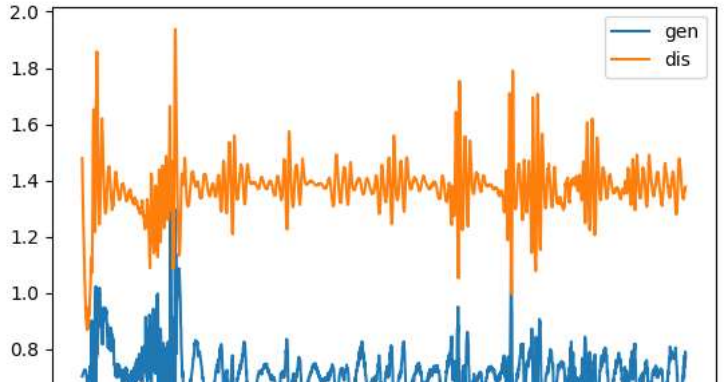
---------- epoch 49 ----------

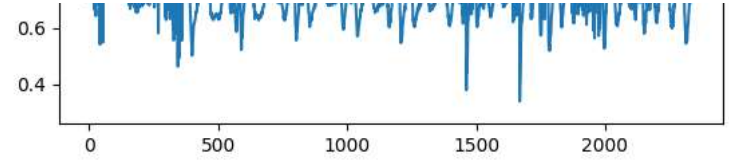


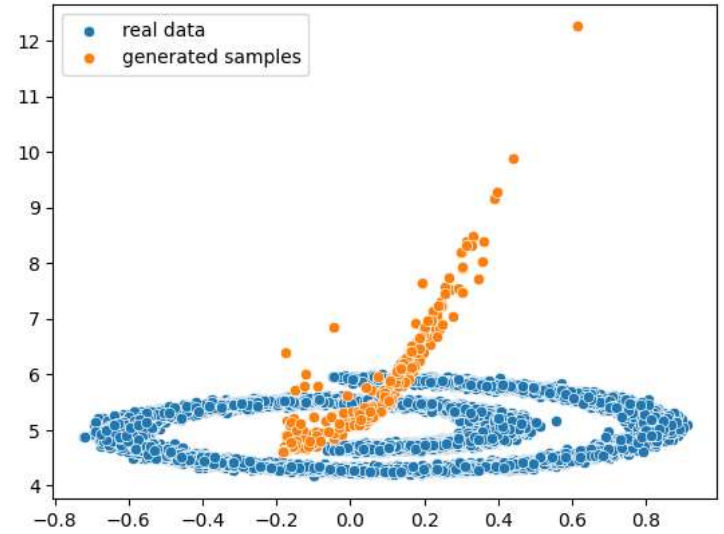

1.6
1.4
1.2
1.0
0.8
0.6
0.4
0
250
500
750
1000
1250
1500
1750
2000


real data
generated samples
8
7
6
5
4
−1.0
−0.5
0.0
0.5
1.0

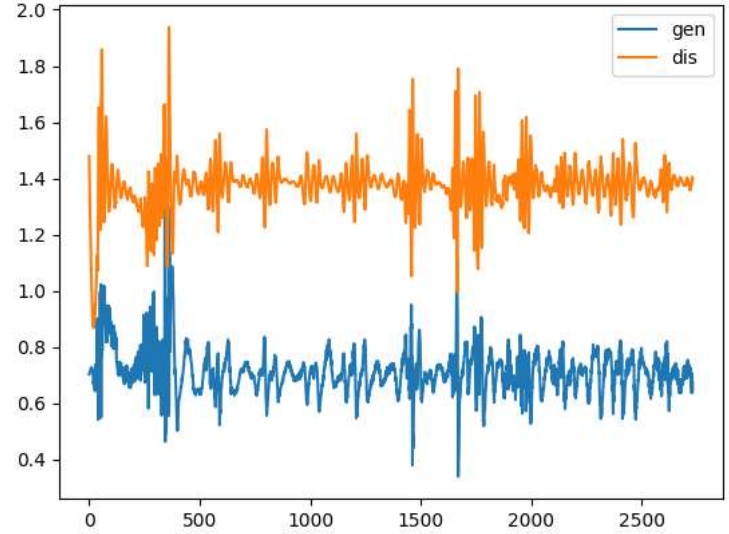
epoch 59


gen
dis
2.0
1.8
1.6
1.4
1.2
1.0
0.8

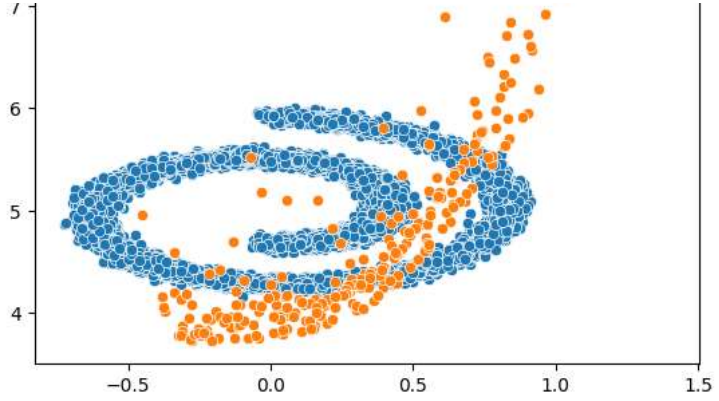

0.6
0.4
0
500
1000
1500
2000


real data
generated samples
12
11
10
9
8
7
6
5
4
−0.8
−0.6
−0.4
−0.2
0.0
0.2
0.4
0.6
0.8

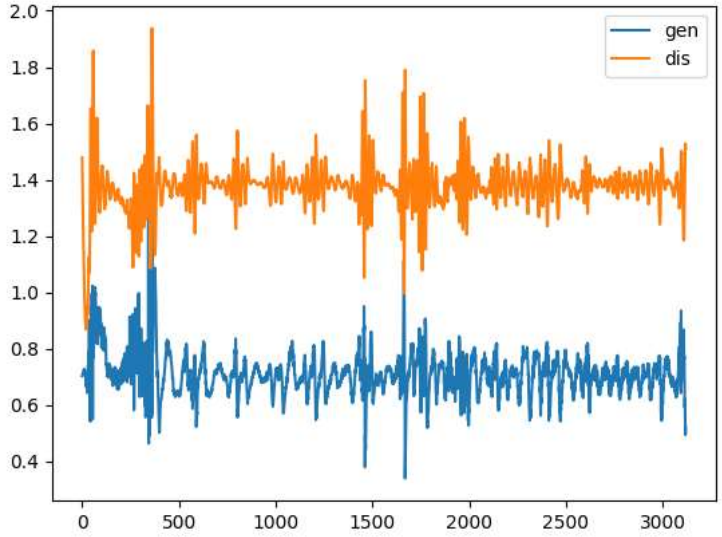
epoch 69

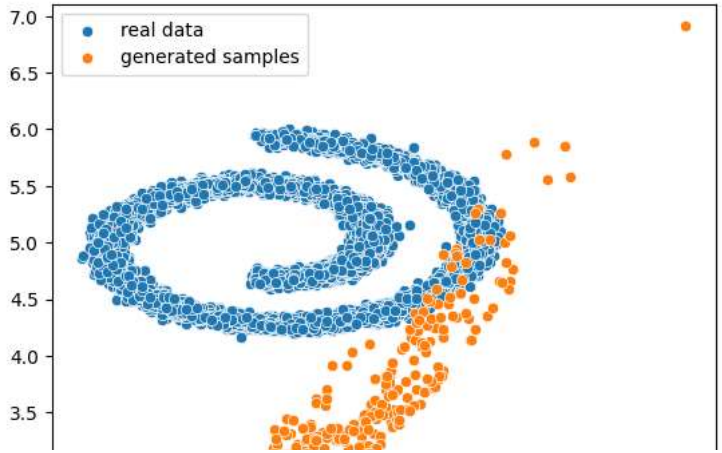

gen
dis
2.0
1.8
1.6
1.4
1.2
1.0
0.8
0.6
0.4
0
500
1000
1500
2000
2500

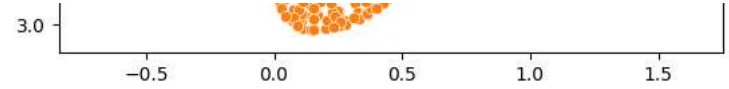

real data
generated samples
8


7
6
5
4
−0.5
0.0
0.5
1.0
1.5

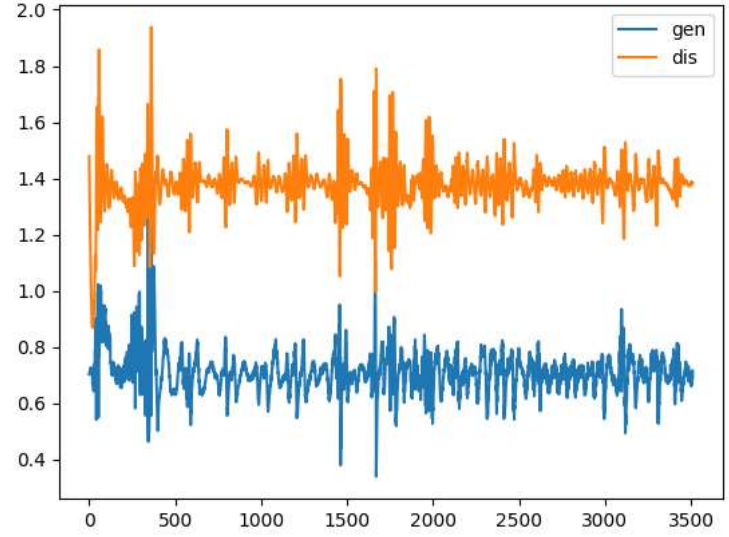
---------- epoch 79 ----------

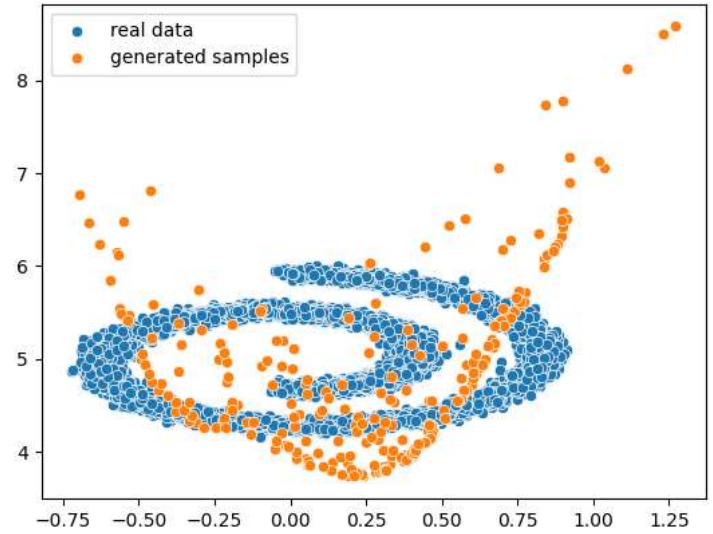

gen
dis
2.0
1.8
1.6
1.4
1.2
1.0
0.8
0.6
0.4
0
500
1000
1500
2000
2500
3000


real data
generated samples
7.0
6.5
6.0
5.5
5.0
4.5
4.0
3.5

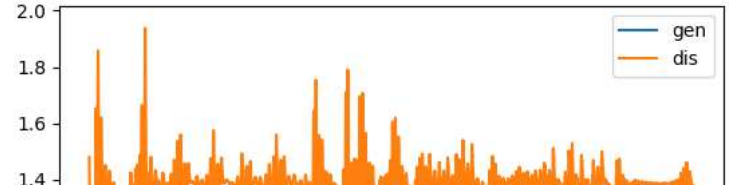


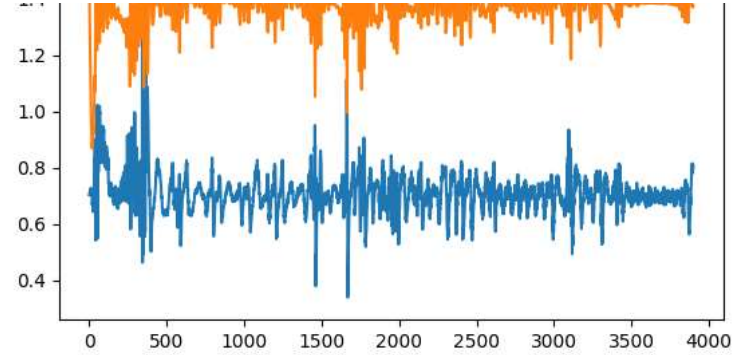
---------- epoch 89 ----------

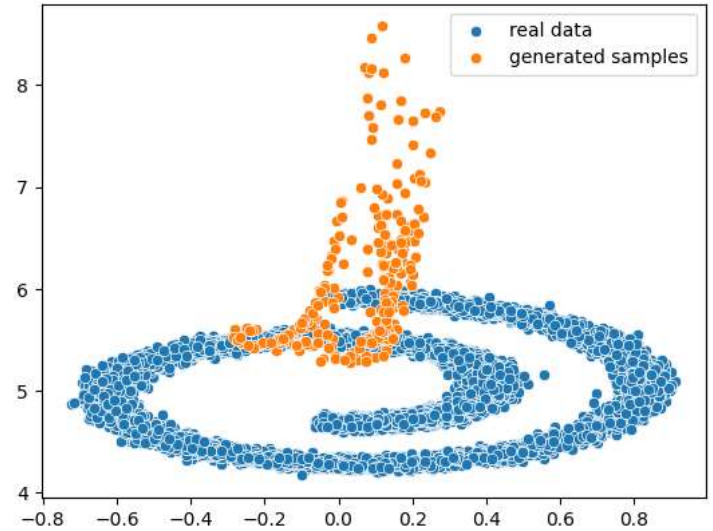




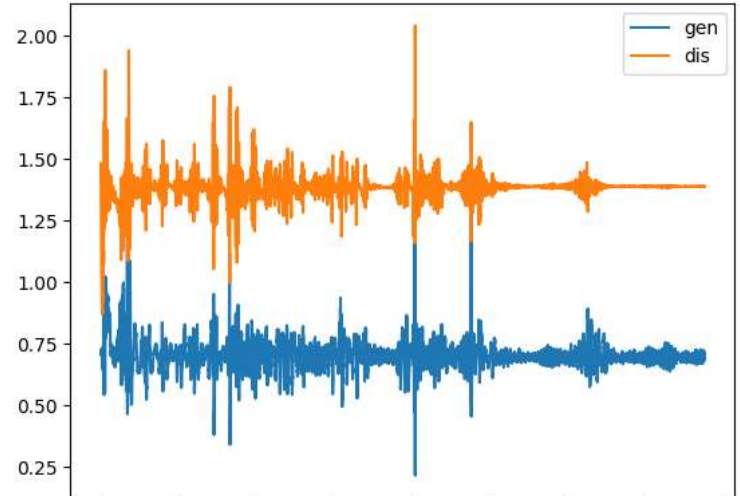
---------- epoch 99 ----------

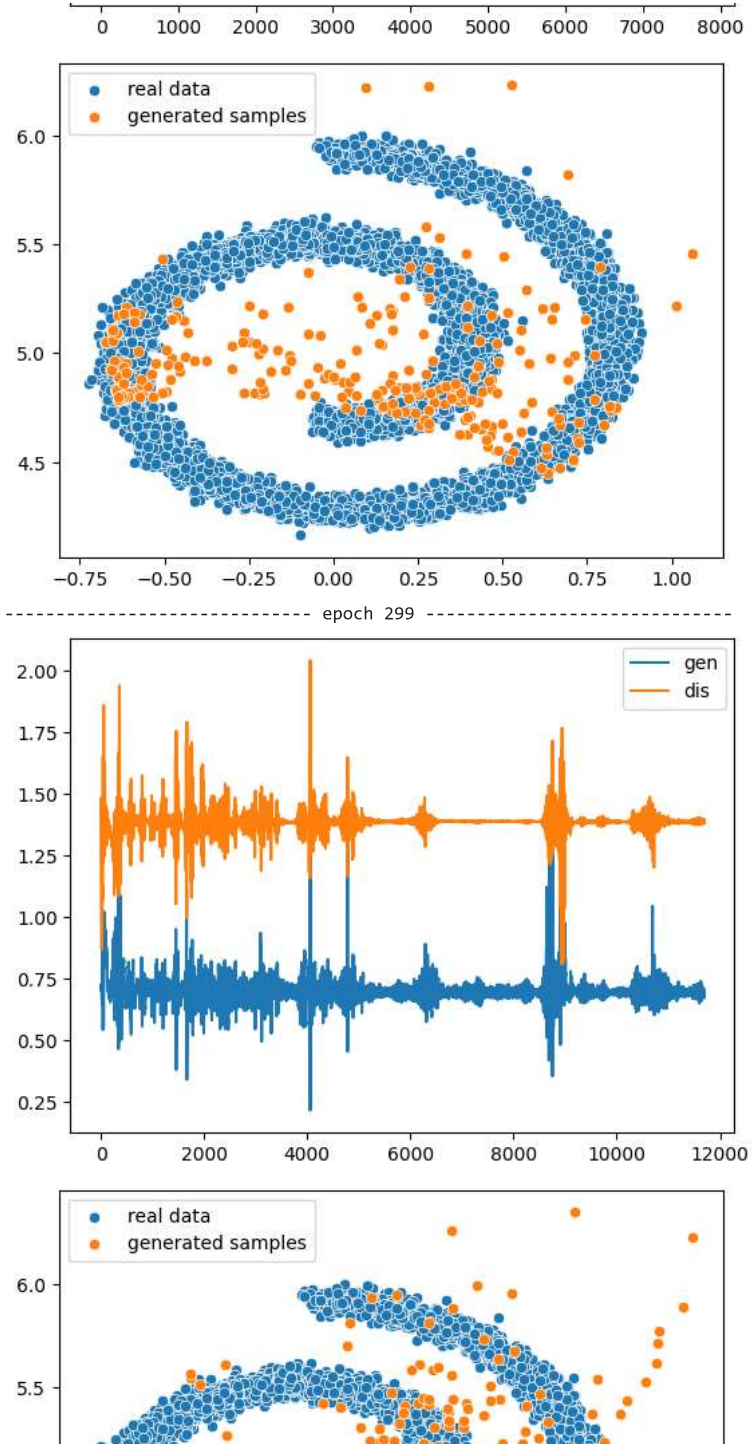


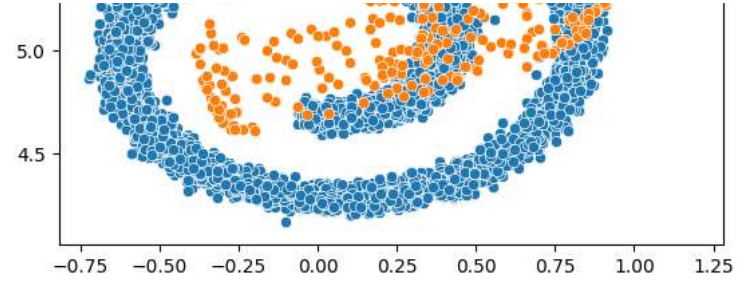

1.2
1.0
0.8
0.6
0.4
0
500
1000
1500
2000
2500
3000
3500
4000


real data
generated samples
8
7
6
5
4
−0.8
−0.6
−0.4
−0.2
0.0
0.2
0.4
0.6
0.8

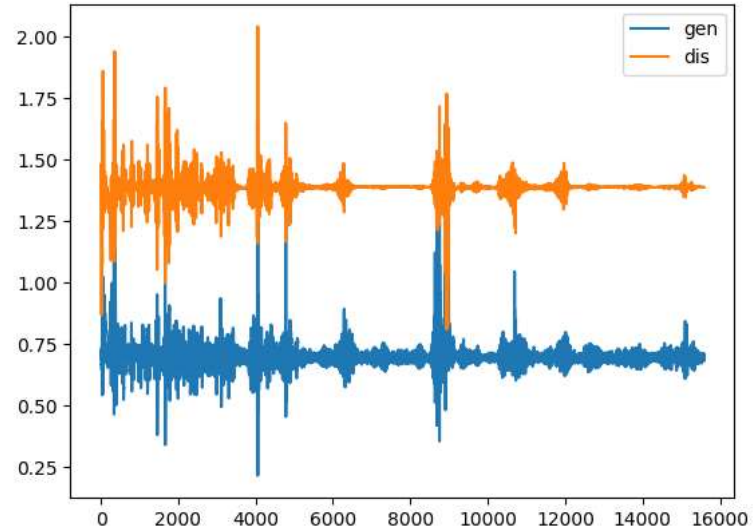
epoch 199

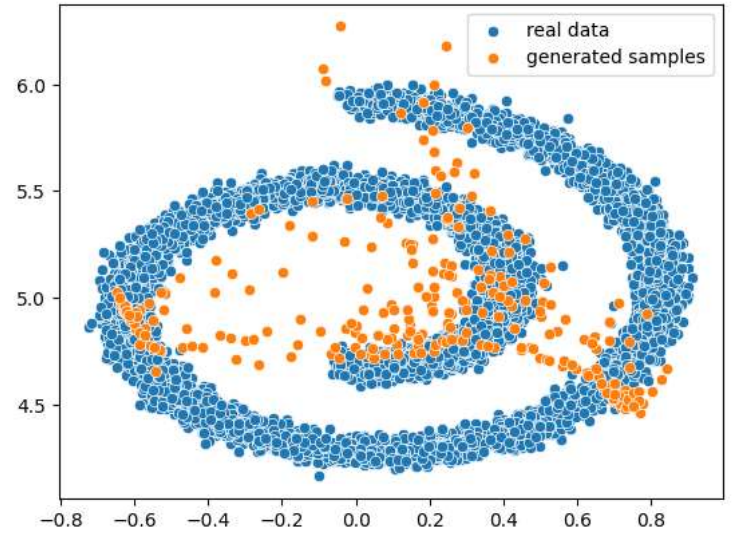

gen
dis
2.00
1.75
1.50
1.25
1.00
0.75
0.50
0.25


0
1000
2000
3000
4000
5000
6000
7000
8000
real data
generated samples
6.0
5.5
5.0
4.5
−0.75
−0.50
−0.25
0.00
0.25
0.50
0.75
1.00
epoch 299
gen
dis
2.00
1.75
1.50
1.25
1.00
0.75
0.50
0.25
0
2000
4000
6000
8000
10000
12000
real data
generated samples
6.0
5.5

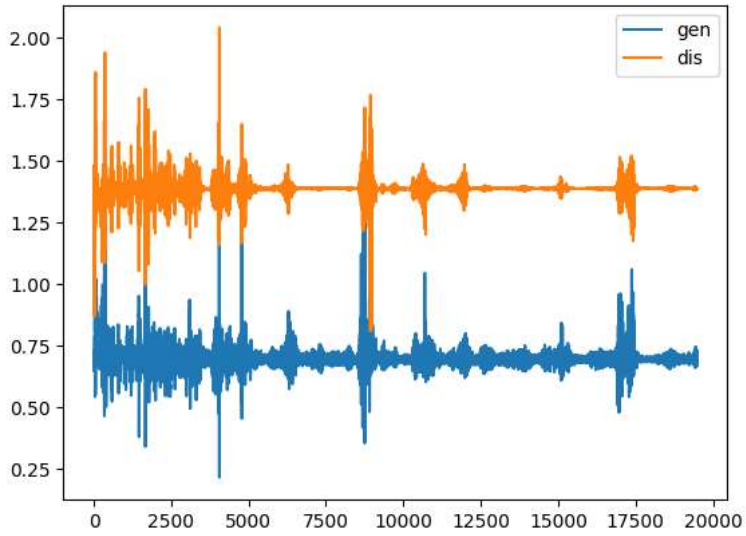


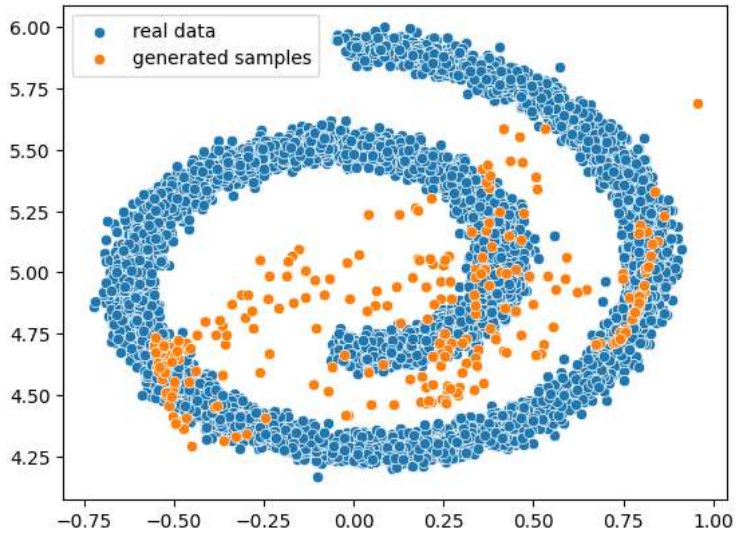
---------- epoch 399 ----------



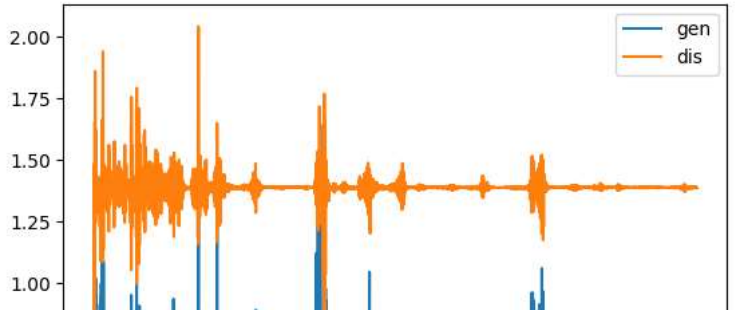


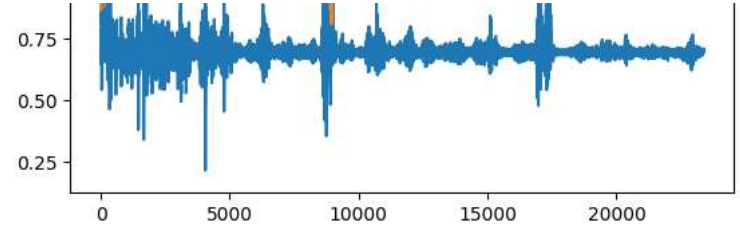
---------- epoch 499 ----------

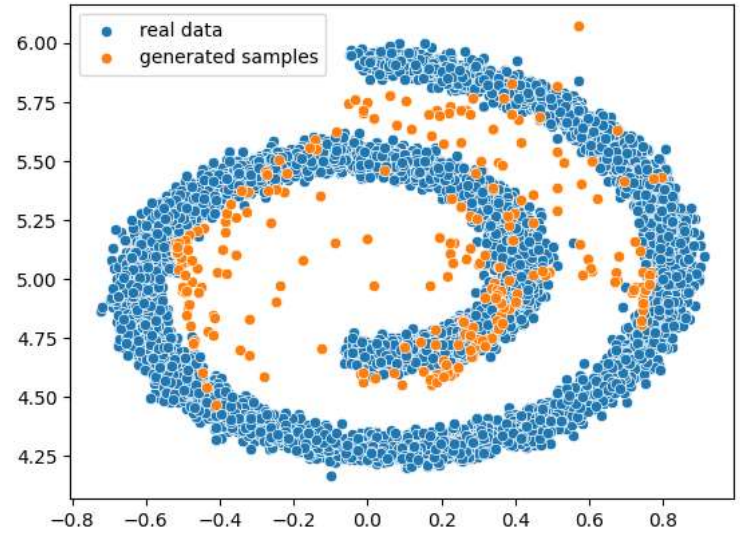

gen
dis
2.00
1.75
1.50
1.25
1.00
0.75
0.50
0.25
0
2500
5000
7500
10000
12500
15000
17500
20000


real data
generated samples
6.00
5.75
5.50
5.25
5.00
4.75
4.50
4.25
−0.75
−0.50
−0.25
0.00
0.25
0.50
0.75
1.00

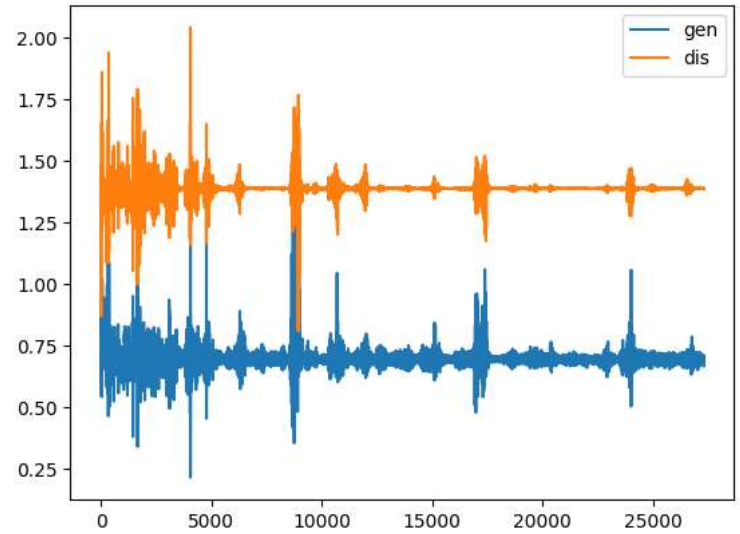
---------- epoch 599 ----------


gen
dis
2.00
1.75
1.50
1.25
1.00

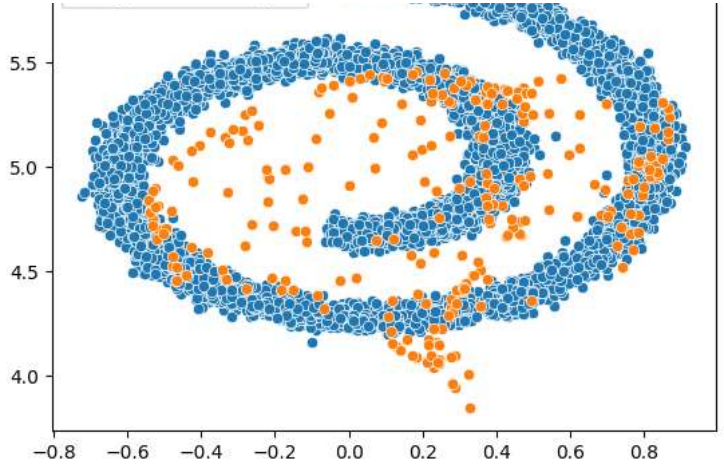

0.75
0.50
0.25
0
5000
10000
15000
20000


real data
generated samples
6.00
5.75
5.50
5.25
5.00
4.75
4.50
4.25
−0.8
−0.6
−0.4
−0.2
0.0
0.2
0.4
0.6
0.8

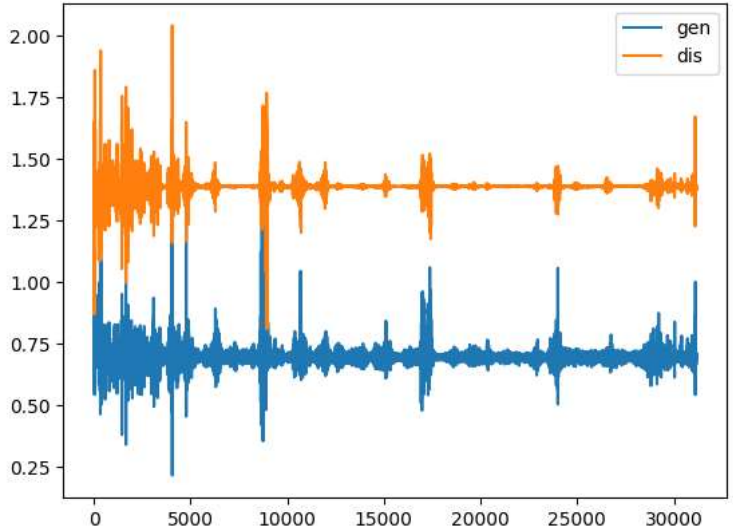
---------- epoch 699 ----------

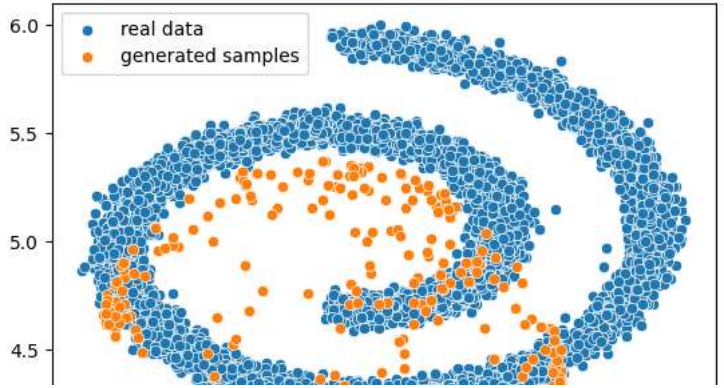

gen
dis
2.00
1.75
1.50
1.25
1.00
0.75
0.50
0.25
0
5000
10000
15000
20000
25000

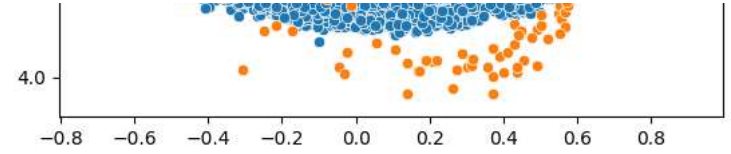

real data
generated samples
6.0


5.5
5.0
4.5
4.0
−0.8
−0.6
−0.4
−0.2
0.0
0.2
0.4
0.6
0.8

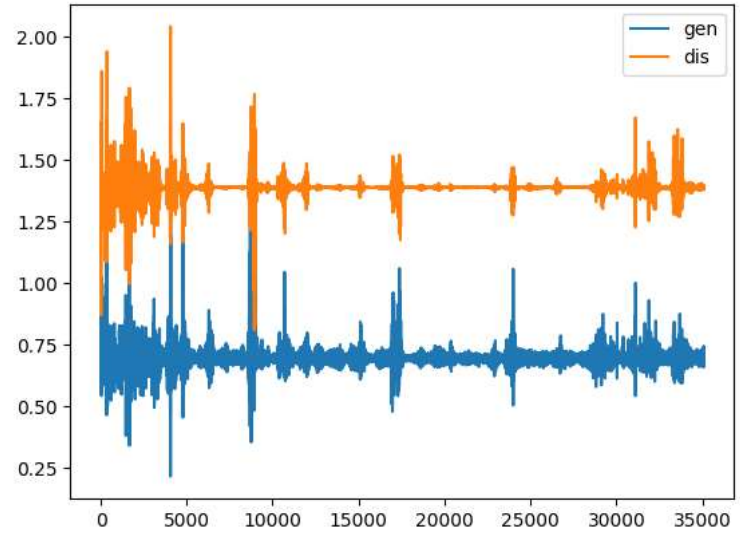
epoch 799

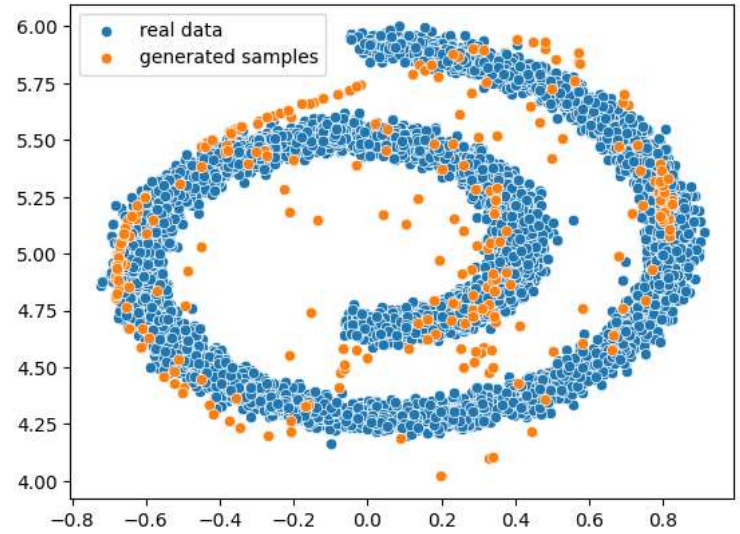

gen
dis
2.00
1.75
1.50
1.25
1.00
0.75
0.50
0.25
0
5000
10000
15000
20000
25000
30000


real data
generated samples
6.0
5.5
5.0
4.5



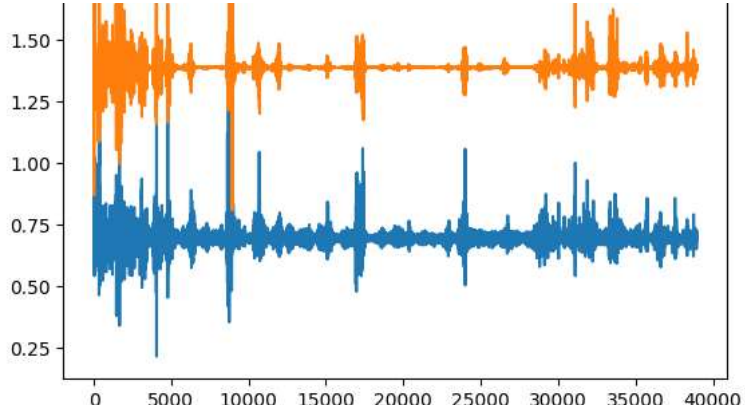
epoch 899





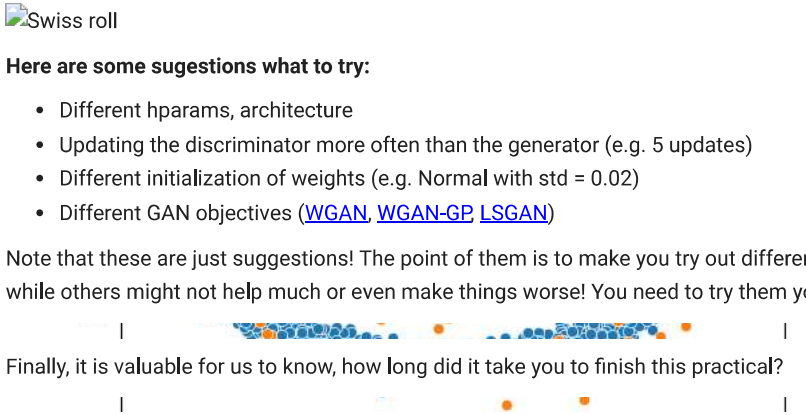
epoch 999





As you see the model (with the original hyperparameter values) is not really able to model the data well :/

Are you able to improve the model to get a results like the following figure?



Here are some sugestions what to try:

- Different hparams, architecture
- Updating the discriminator more often than the generator (e.g. 5 updates)
- Different initialization of weights (e.g. Normal with std = 0.02)
- Different GAN objectives (WGAN, WGAN-GP, LSGAN)

Note that these are just suggestions! The point of them is to make you try out different things yourself. Some of them should help significantly, while others might not help much or even make things worse! You need to try them yourself to find out!

Finally, it is valuable for us to know, how long did it take you to finish this practical?